Akita: A High-Performance Lattice-Based PCS



Quang Dao
CMU

Joint work with: Omid Bodaghi, Giuseppe Vitto, Amirhossein Khajehpour, Taghi Badakhshan (LayerZero Labs), Fengrun Liu (CMU), Jiapeng Zhang (USC), Markos Georghiades (a16z crypto), and Justin Thaler (a16z crypto & Georgetown)

NYU CryptoSec Seminar, October 2026

From Signatures to SNARKs

From Signatures to SNARKs

Digital signature



"I know the secret key"

a **fixed** statement, tied to the message

From Signatures to SNARKs

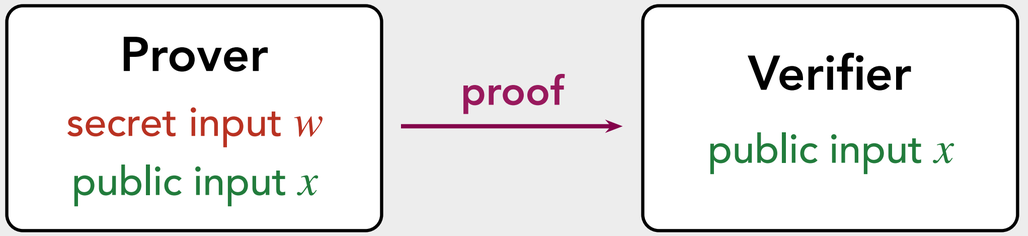
Digital signature



"I know the secret key"

a **fixed** statement, tied to the message

SNARK for a program or circuit P



"I know w such that $P(x, w)$ accepts"

any statement: P can be arbitrary

From Signatures to SNARKs

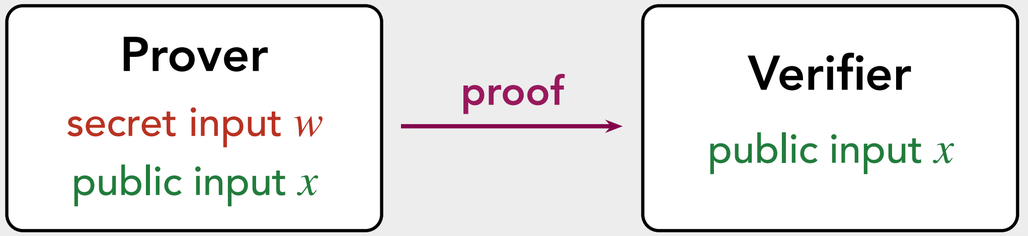
Digital signature



"I know the secret key"

a **fixed** statement, tied to the message

SNARK for a program or circuit P



"I know w such that $P(x, w)$ accepts"

any statement: P can be arbitrary

What (zk)SNARKs are used for

SNARK: Succinct Non-interactive ARgument of Knowledge

Scaling blockchains

rollups, zkVMs

compress validity of many transactions into one proof

Private payments

Zcash, Aztec

a valid payment, hiding who paid whom

Digital identity

Google Wallet, EU wallet

prove you are over 18 and reveal nothing else

Verifiable AI

Attestable, Lagrange, EZKL

prove which model produced an answer

SNARK = PIOP + PCS

SNARK = PIOP + PCS

Prover Verifier

PIOP

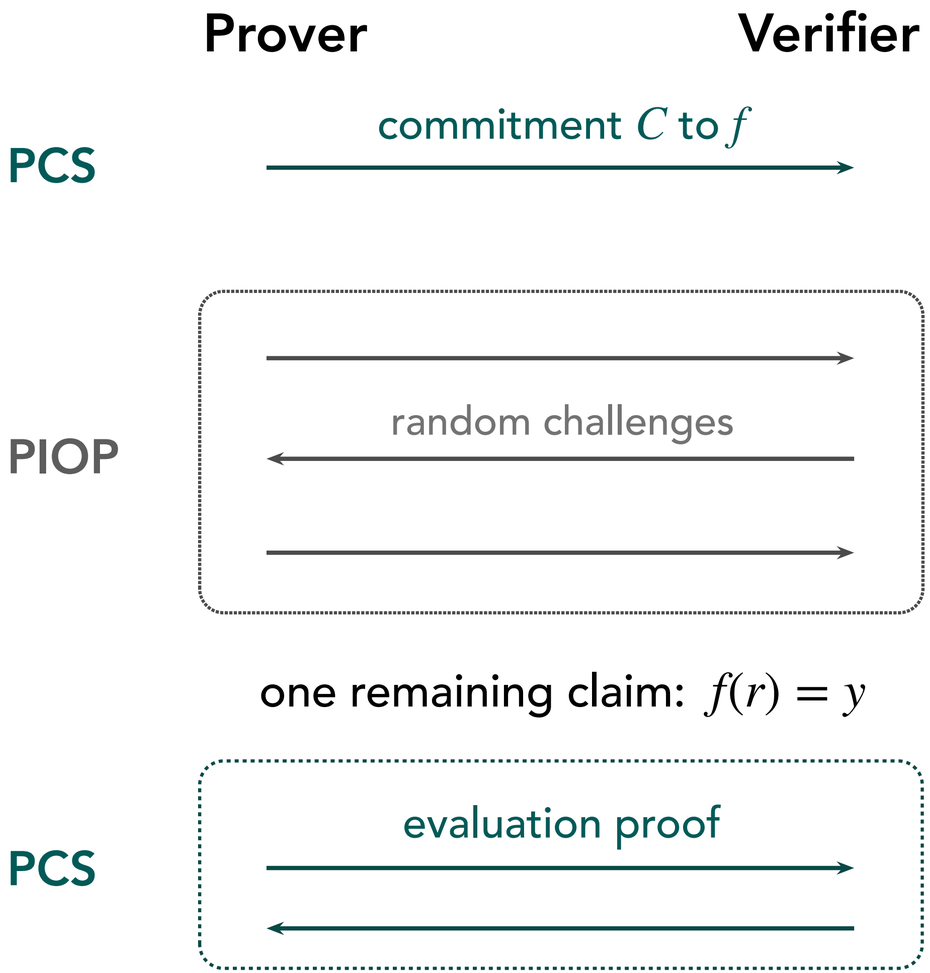


one remaining claim: $f(r) = y$

Polynomial IOP (PIOP)

- the prover sends polynomials $f, \ldots$ that encode the run
- the verifier checks them at a few random points
- **statistically sound**: no cryptographic assumption

SNARK = PIOP + PCS



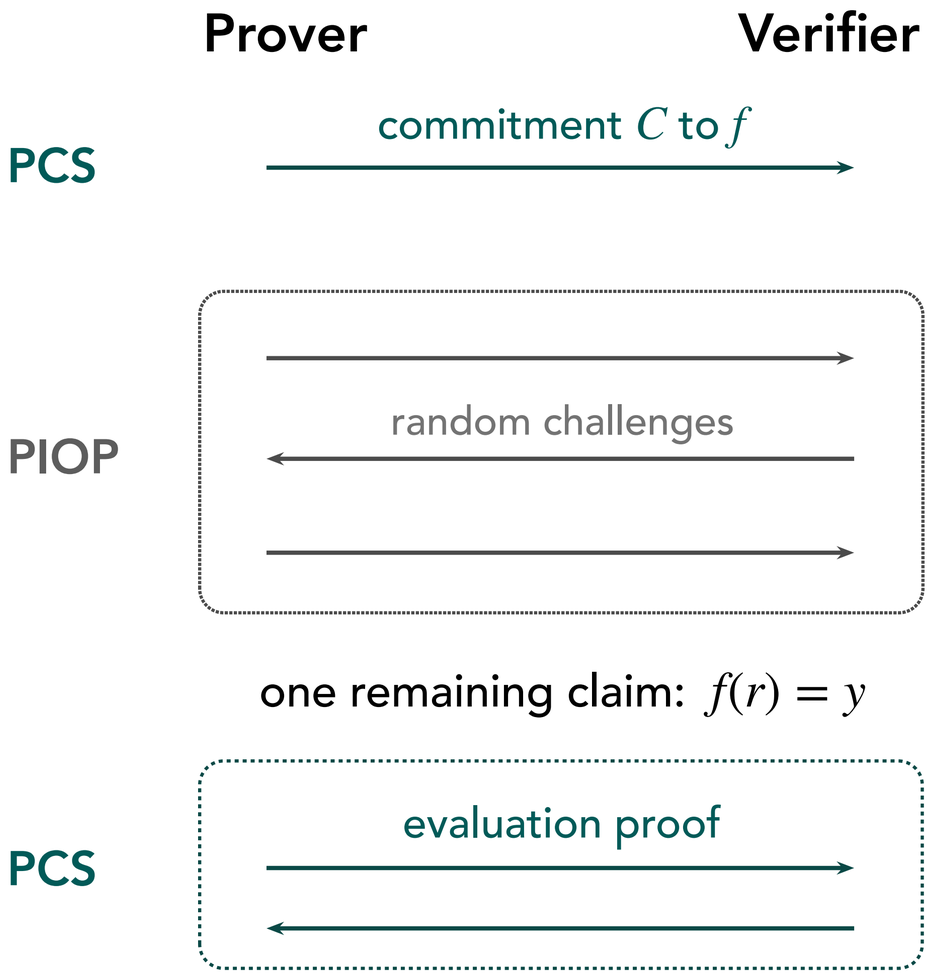
Polynomial IOP (PIOP)

- the prover sends polynomials $f, \ldots$ that encode the run
- the verifier checks them at a few random points
- **statistically sound:** no cryptographic assumption

Polynomial commitment scheme (PCS)

- a polynomial f is often too large to send in the clear
- **commit:** a short C stands in for f
- **open:** a short proof that $f(r) = y$
- **soundness** rests on a cryptographic assumption

SNARK = PIOP + PCS



Polynomial IOP (PIOP)

- the prover sends polynomials $f, \ldots$ that encode the run
- the verifier checks them at a few random points
- **statistically sound:** no cryptographic assumption

Polynomial commitment scheme (PCS)

- a polynomial f is often too large to send in the clear
- **commit:** a short C stands in for f
- **open:** a short proof that $f(r) = y$
- **soundness** rests on a cryptographic assumption

All cryptographic assumptions are in the PCS: a post-quantum PCS gives a post-quantum SNARK

Post-Quantum Migration: Lattices versus Hashes

Post-Quantum Migration: Lattices versus Hashes

	elliptic curves	hashes	lattices
Signatures	Ed25519: 64 bytes not post-quantum	SLH-DSA: 8 to 50 KB NIST standard	ML-DSA, Falcon: 0.7 to 4.6 KB ML-DSA: NIST's primary standard

Post-Quantum Migration: Lattices versus Hashes

	elliptic curves	hashes	lattices
Signatures	Ed25519: 64 bytes not post-quantum	SLH-DSA: 8 to 50 KB NIST standard	ML-DSA, Falcon: 0.7 to 4.6 KB ML-DSA: NIST's primary standard
SNARKs	Groth16: 128 bytes not post-quantum	FRI, WHIR: 150 KB and up deployed: StarkWare, Succinct, …	Greyhound [NS24]: 53 KB not deployed anywhere

Post-Quantum Migration: Lattices versus Hashes

	elliptic curves	hashes	lattices
Signatures	Ed25519: 64 bytes not post-quantum	SLH-DSA: 8 to 50 KB NIST standard	ML-DSA, Falcon: 0.7 to 4.6 KB ML-DSA: NIST's primary standard
SNARKs	Groth16: 128 bytes not post-quantum	FRI, WHIR: 150 KB and up deployed: StarkWare, Succinct, …	Greyhound [NS24]: 53 KB not deployed anywhere
Assumption	discrete log, pairings	a hash function, as a random oracle	Module-SIS, Module-LWE linear algebra with small numbers

Post-Quantum Migration: Lattices versus Hashes

	elliptic curves	hashes	lattices
Signatures	Ed25519: 64 bytes not post-quantum	SLH-DSA: 8 to 50 KB NIST standard	ML-DSA, Falcon: 0.7 to 4.6 KB ML-DSA: NIST's primary standard
SNARKs	Groth16: 128 bytes not post-quantum	FRI, WHIR: 150 KB and up deployed: StarkWare, Succinct, …	Greyhound [NS24]: 53 KB not deployed anywhere
Assumption	discrete log, pairings	a hash function, as a random oracle	Module-SIS, Module-LWE linear algebra with small numbers

Lattice-based signatures are the default. Why not lattice-based SNARKs?

Why Lattice-Based PCSs Are Not Deployment-Ready

Why Lattice-Based PCSs Are Not Deployment-Ready

	proof size	verifier at $N = 2^{30}$	assumption	code
Greyhound [NS24]	53 KB	$\sqrt{N}$: 2.8 s	Module-SIS	AVX-512 only
Hachi [NOZ26]	55 KB, estimated	$\sqrt{N}$: 227 ms, estimated	Module-SIS	one fold only

Why Lattice-Based PCSs Are Not Deployment-Ready

	proof size	verifier at $N = 2^{30}$	assumption	code
Greyhound [NS24]	53 KB	$\sqrt{N}$: 2.8 s	Module-SIS	AVX-512 only
Hachi [NOZ26]	55 KB, estimated	$\sqrt{N}$: 227 ms, estimated	Module-SIS	one fold only
Jindo [HLSS26]	1.2 MB at 2^{20}	$N^{1/3}$: 14 ms at 2^{20}	Module-SIS*	yes
RoKoKo [KLNOT26]	157 KB	polylog: 13 ms	vanishing SIS*	yes
Grand Danois [KK26]	80 KB, estimated	polylog: n/a	vanishing SIS	none
Maltese [CNT26]	335 KB, estimated	polylog: n/a	Module-SIS	none

* with a heuristic challenge set

Why Lattice-Based PCSs Are Not Deployment-Ready

	proof size	verifier at $N = 2^{30}$	assumption	code
Greyhound [NS24]	53 KB	$\sqrt{N}$: 2.8 s	Module-SIS	AVX-512 only
Hachi [NOZ26]	55 KB, estimated	$\sqrt{N}$: 227 ms, estimated	Module-SIS	one fold only
Jindo [HLSS26]	1.2 MB at 2^{20}	$N^{1/3}$: 14 ms at 2^{20}	Module-SIS*	yes
RoKoKo [KLNOT26]	157 KB	polylog: 13 ms	vanishing SIS*	yes
Grand Danois [KK26]	80 KB, estimated	polylog: n/a	vanishing SIS	none
Maltese [CNT26]	335 KB, estimated	polylog: n/a	Module-SIS	none

* with a heuristic challenge set

Deployment requires more than just performance:

- one proof for commitments made at different times; proving across many machines
- code that withstands review: mistakes were found, and since fixed, in LaBRADOR, Greyhound, RoKoKo

Why Lattice-Based PCSs Are Not Deployment-Ready

	proof size	verifier at $N = 2^{30}$	assumption	code
Greyhound [NS24]	53 KB	$\sqrt{N}$: 2.8 s	Module-SIS	AVX-512 only
Hachi [NOZ26]	55 KB, estimated	$\sqrt{N}$: 227 ms, estimated	Module-SIS	one fold only
Jindo [HLSS26]	1.2 MB at 2^{20}	$N^{1/3}$: 14 ms at 2^{20}	Module-SIS*	yes
RoKoKo [KLNOT26]	157 KB	polylog: 13 ms	vanishing SIS*	yes
Grand Danois [KK26]	80 KB, estimated	polylog: n/a	vanishing SIS	none
Maltese [CNT26]	335 KB, estimated	polylog: n/a	Module-SIS	none

* with a heuristic challenge set

Deployment requires more than just performance:

- one proof for commitments made at different times; proving across many machines
- code that withstands review: mistakes were found, and since fixed, in LaBRADOR, Greyhound, RoKoKo

Can a lattice-based PCS have small proofs, a fast verifier, and security from Module-SIS?

Akita: A Production-Ready Lattice-Based PCS

Akita: A Production-Ready Lattice-Based PCS

Opening a polynomial with 2^{24} to 2^{30} coefficients 32-bit field, one thread, AMD Ryzen 9 9950X

58 to 64 KB
proofs

7 to 14 ms
verification

Module-SIS
standard assumption

Akita: A Production-Ready Lattice-Based PCS

Opening a polynomial with 2^{24} to 2^{30} coefficients 32-bit field, one thread, AMD Ryzen 9 9950X

58 to 64 KB

proofs

Greyhound*: 60 to 67 KB

7 to 14 ms

verification

Greyhound*: 148 to 1,537 ms

Module-SIS

standard assumption

* when tuned to the same lattice security level as Akita

Akita: A Production-Ready Lattice-Based PCS

Opening a polynomial with 2^{24} to 2^{30} coefficients 32-bit field, one thread, AMD Ryzen 9 9950X

58 to 64 KB

proofs

Greyhound*: 60 to 67 KB

7 to 14 ms

verification

Greyhound*: 148 to 1,537 ms

Module-SIS

standard assumption

1. Setup offloading

verifier: $\sqrt{N} \rightarrow N^{1/K}$

the prover applies the commitment matrix, and proves it

verification	32 ms ⟶	**14 ms**
proof size	61 KB ⟶	**64 KB**
opening time	5.7 s ⟶	**8.4 s**



with and without offloading, at 2^{30} coefficients

* when tuned to the same lattice security level as Akita

Akita: A Production-Ready Lattice-Based PCS

Opening a polynomial with 2^{24} to 2^{30} coefficients 32-bit field, one thread, AMD Ryzen 9 9950X

58 to 64 KB
proofs
Greyhound*: 60 to 67 KB

7 to 14 ms
verification
Greyhound*: 148 to 1,537 ms

Module-SIS
standard assumption

1. Setup offloading

verifier: $\sqrt{N} \to N^{1/K}$

the prover applies the commitment matrix, and proves it

verification	32 ms ⟶	**14 ms**
proof size	61 KB ⟶	**64 KB**
opening time	5.7 s ⟶	**8.4 s**



with and without offloading, at 2^{30} coefficients

2. A leaner fold

each fold is cheaper to prove, smaller to send

commitment	1 to 8 KB ⟶	**128 bytes**
range check, prover	$O(b^2L)$ ⟶	$\boldsymbol{O(bL)}$
evaluation check	1 element ⟶	**none sent**

compared with the folds of Greyhound and Hachi

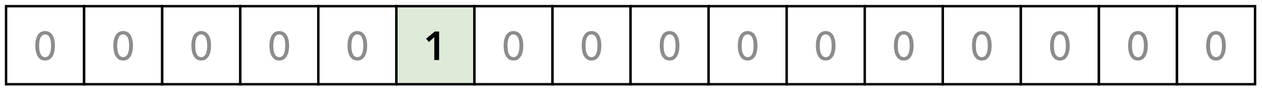
* when tuned to the same lattice security level as Akita

First Application: Post-Quantum Jolt

First Application: Post-Quantum Jolt

Jolt is a zkVM: a SNARK for "this RISC-V program ran correctly"

What Jolt commits to: one-hot polynomials

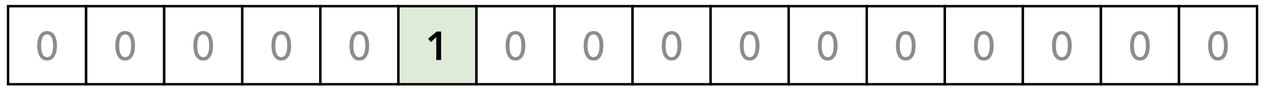


a single 1 per step of the program

First Application: Post-Quantum Jolt

Jolt is a zkVM: a SNARK for "this RISC-V program ran correctly"

What Jolt commits to: one-hot polynomials



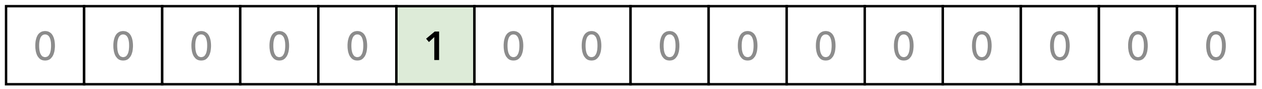
a single 1 per step of the program

- **Dory, in Jolt today:** pays only for the 1s, but rests on elliptic curves
- **a hash-based PCS:** post-quantum, but pays for every 0
- **Akita:** pays only for the 1s, and is post-quantum

First Application: Post-Quantum Jolt

Jolt is a zkVM: a SNARK for "this RISC-V program ran correctly"

What Jolt commits to: one-hot polynomials



a single 1 per step of the program

- **Dory, in Jolt today:** pays only for the 1s, but rests on elliptic curves
- **a hash-based PCS:** post-quantum, but pays for every 0
- **Akita:** pays only for the 1s, and is post-quantum

Jolt with Akita in place of Dory

2.2 to 4.2×
faster prover

2.8 to 8.5×
faster verifier

under 100 KB
proofs

Roadmap

1. The Lattice-Based PCS Template
2. Akita and Its Core Techniques
3. Evaluation & Future Work

Roadmap

1. The Lattice-Based PCS Template
2. Akita and Its Core Techniques
3. Evaluation & Future Work

Polynomial Commitment Schemes

Polynomial Commitment Schemes

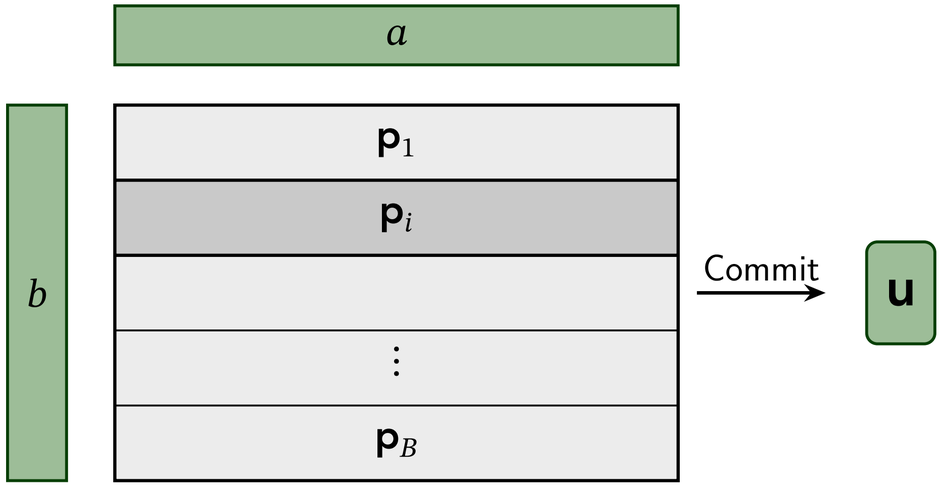
p: a multilinear polynomial, $N = 2^{\mu}$ evaluations in $\{0,1\}^{\mu} \subset \mathbb{F}_q^{\mu}$

private, and far too large to send

Public: a short commitment $\mathbf{u}$, a point x, a value y

Claim: $p(x) = y$ **and** $\mathbf{u} = \mathbf{C}\text{ommit}(p)$

Succinctness: proof size and verifier time far below N



the table of p: B rows of length M

Polynomial Commitment Schemes

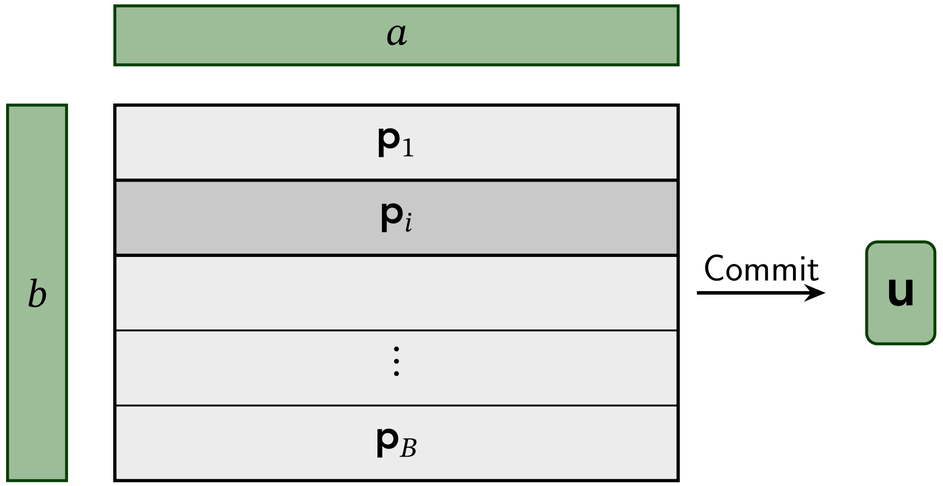
p: a multilinear polynomial, $N = 2^\mu$ evaluations in $\{0,1\}^\mu \subset \mathbb{F}_q^\mu$

private, and far too large to send

Public: a short commitment $\mathbf{u}$, a point x, a value y

Claim: $\boldsymbol{p(x) = y}$ **and** $\mathbf{u} = \mathbf{C}\text{ommit}(\boldsymbol{p})$

Succinctness: proof size and verifier time far below N



the table of p: B rows of length M

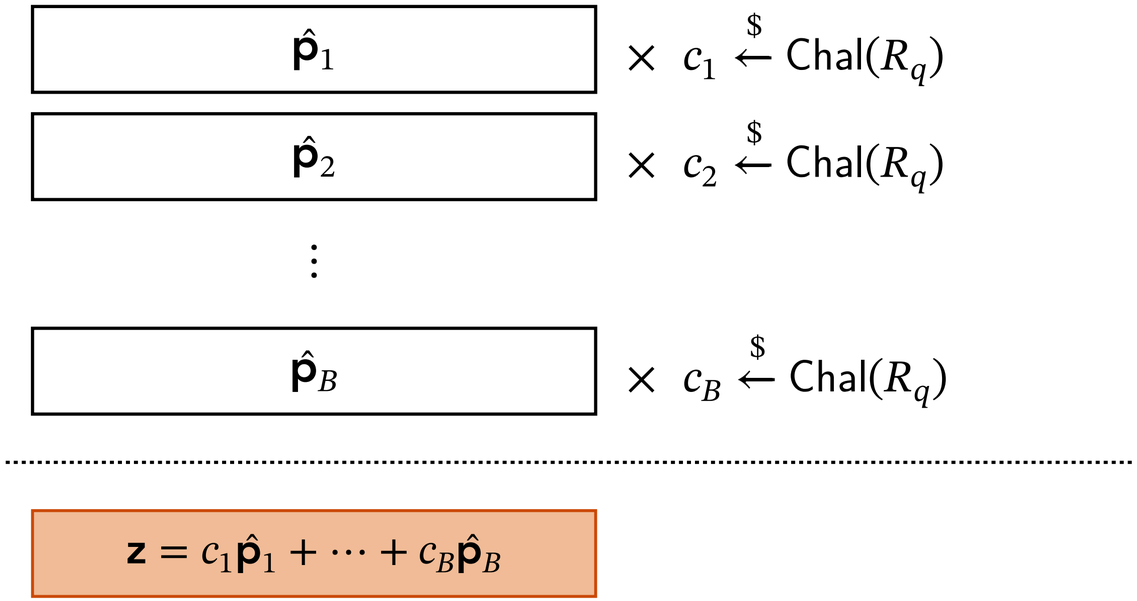
Re-size p into a table, and leverage vector-matrix-vector form:

- the weights a and b are determined by the halves of x: $a = \bigotimes_j (1 - x_{L,j}, x_{L,j})$, $b = \bigotimes_i (1 - x_{R,i}, x_{R,i})$
- each row has a partial evaluation $e_i = \langle \mathbf{p}_i, a \rangle$, and $y = \sum_i b_i \cdot e_i$

Succinctness by Folding

Succinctness by Folding

Transform p into blocks of ring elements, folded into one using short randomness



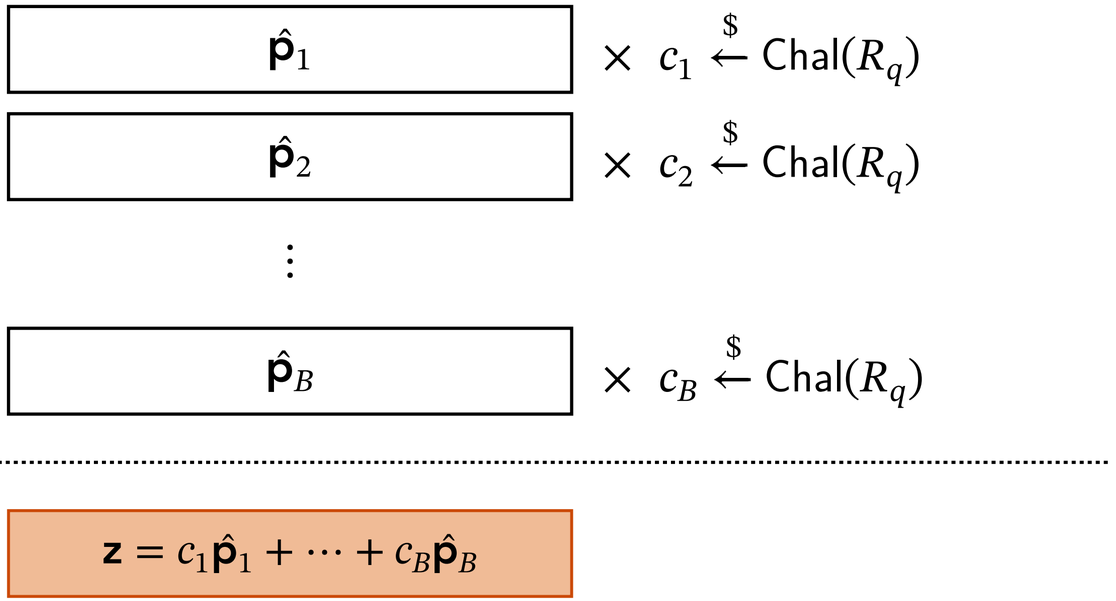
$R_q := \mathbb{F}_q[X]/(X^d + 1)$ **is the cyclotomic ring with** $d = 2^k$

$\mathsf{Chal}(R_q)$ **is the ring challenge set: low norm, invertible pairwise differences**

($d = 128$: 31 random entries are -1 or 1, the rest are zero; R_q must not split too much)

Succinctness by Folding

Transform p into blocks of ring elements, folded into one using short randomness



- Digit-decompose each row $\mathbf{p}_i$ into $(\hat{\mathbf{p}}_{i,j})_{j\in[\ell]}$
- B blocks of length $\ell \cdot M$, in short digits
- the verifier picks short ring challenges $c_1, \ldots, c_B$
- $\mathbf{z}$ has the length of one block, and is still short

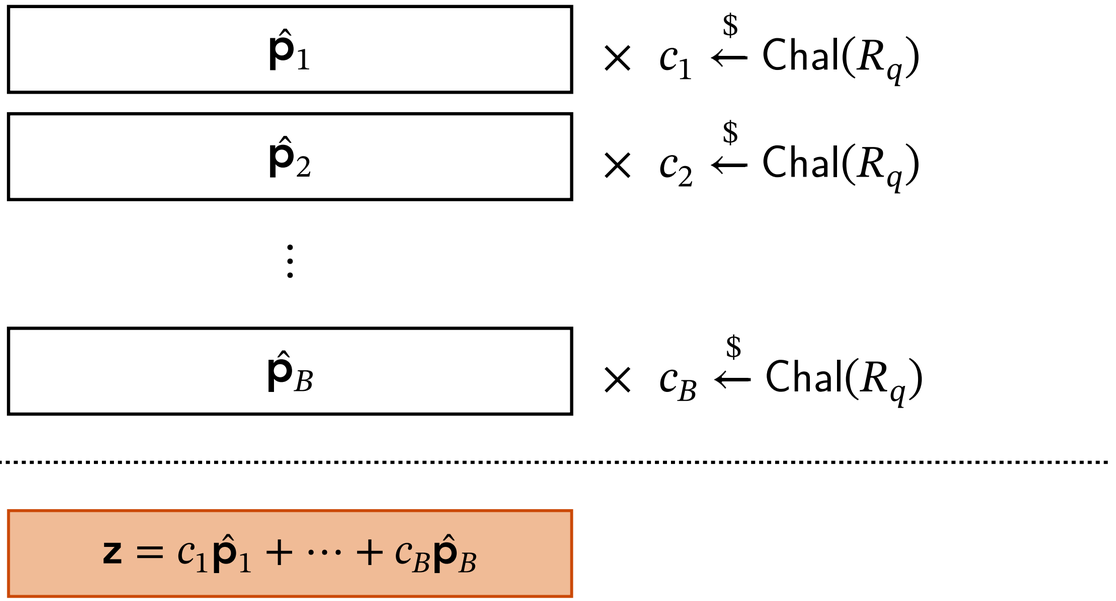
$R_q := \mathbb{F}_q[X]/(X^d + 1)$ **is the cyclotomic ring with** $d = 2^k$

$\mathrm{Chal}(R_q)$ **is the ring challenge set: low norm, invertible pairwise differences**

($d = 128$: 31 random entries are -1 or 1, the rest are zero; R_q must not split too much)

Succinctness by Folding

Transform p into blocks of ring elements, folded into one using short randomness



- Digit-decompose each row $\mathbf{p}_i$ into $(\hat{\mathbf{p}}_{i,j})_{j\in[\ell]}$
- B blocks of length $\ell \cdot M$, in short digits
- the verifier picks short ring challenges $c_1, \ldots, c_B$
- $\mathbf{z}$ has the length of one block, and is still short

1. **How can we transform p into R_q?**
2. **How can we check $\mathbf{z} \in R_q^m$ is consistent with the PCS relation $p(x) = y$?**

$R_q := \mathbb{F}_q[X]/(X^d + 1)$ is the cyclotomic ring with $d = 2^k$

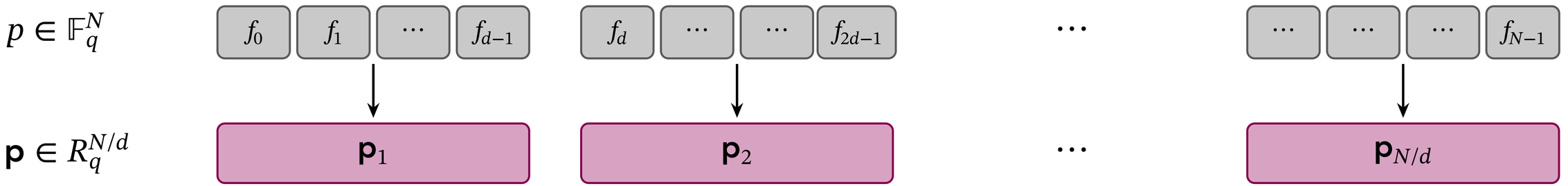
$\mathrm{Chal}(R_q)$ is the ring challenge set: low norm, invertible pairwise differences

($d = 128$: 31 random entries are -1 or 1, the rest are zero; R_q must not split too much)

PCS Evaluation: From Field to Ring

PCS Evaluation: From Field to Ring

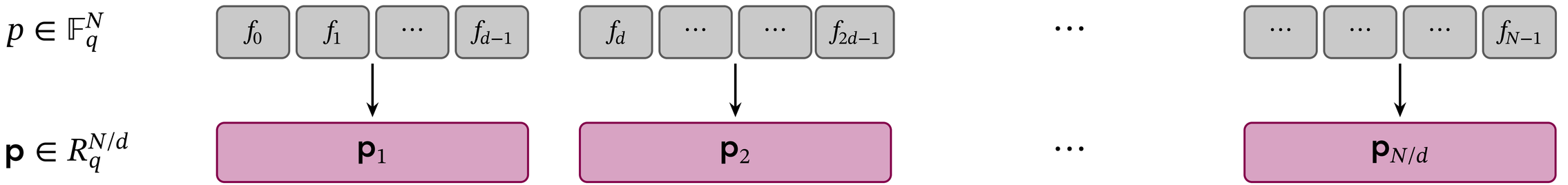
Coefficient packing: d **field values are the coefficients of one ring element**



$\mathbf{p}_1 = f_0 + f_1 X + \ldots + f_{d-1} X^{d-1}$ in $R_q = \mathbb{F}_q[X]/(X^d + 1)$: the same data, d times fewer entries

PCS Evaluation: From Field to Ring

Coefficient packing: d **field values are the coefficients of one ring element**



$\mathbf{p}_1 = f_0 + f_1 X + \ldots + f_{d-1} X^{d-1}$ in $R_q = \mathbb{F}_q[X]/(X^d + 1)$: the same data, d times fewer entries

An inner product over $\mathbb{F}_q$ is the constant term of a ring product:

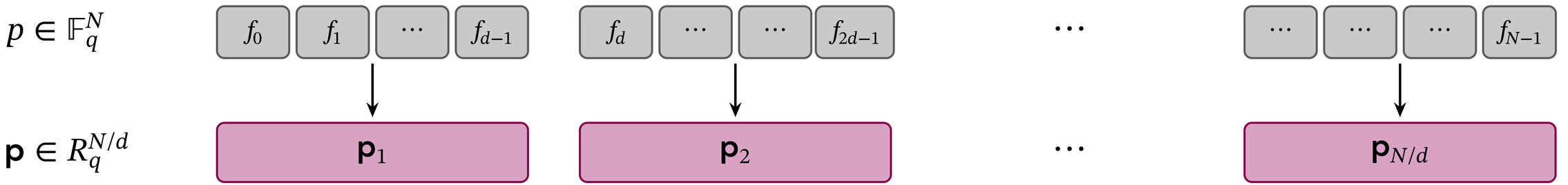
$$\langle f, g \rangle = \mathrm{ct}\big(\mathbf{f} \cdot \sigma_{-1}(\mathbf{g})\big), \qquad \sigma_{-1} : X \mapsto X^{-1}$$

$$\begin{aligned} &\big(f_0 + f_1 X + \ldots + f_{d-1} X^{d-1}\big) \\ &\cdot \big(g_0 - g_{d-1} X + \ldots - g_1 X^{d-1}\big) \\ &= \langle f, g \rangle + \ldots \cdot X + \ldots \end{aligned}$$

since $X^d = -1$

PCS Evaluation: From Field to Ring

Coefficient packing: d field values are the coefficients of one ring element



$\mathbf{p}_1 = f_0 + f_1 X + \ldots + f_{d-1} X^{d-1}$ in $R_q = \mathbb{F}_q[X]/(X^d + 1)$: the same data, d times fewer entries

An inner product over $\mathbb{F}_q$ is the constant term of a ring product:

$$\langle f, g \rangle = \mathrm{ct}\big(\mathbf{f} \cdot \sigma_{-1}(\mathbf{g})\big), \qquad \sigma_{-1} : X \mapsto X^{-1}$$

$$\begin{aligned} &\big(f_0 + f_1 X + \ldots + f_{d-1} X^{d-1}\big) \\ &\cdot \big(g_0 - g_{d-1} X + \ldots - g_1 X^{d-1}\big) \\ &= \langle f, g \rangle + \ldots \cdot X + \ldots \end{aligned}$$

The claim $p(x) = y$ becomes a claim over R_q:

$$\mathbf{e}_i = \langle \mathbf{p}_i, \mathbf{a} \rangle \text{ for each block,} \qquad y = \mathrm{ct}\left(\left(\sum_i b_i \mathbf{e}_i\right) \cdot \sigma_{-1}(\mathbf{o})\right)$$

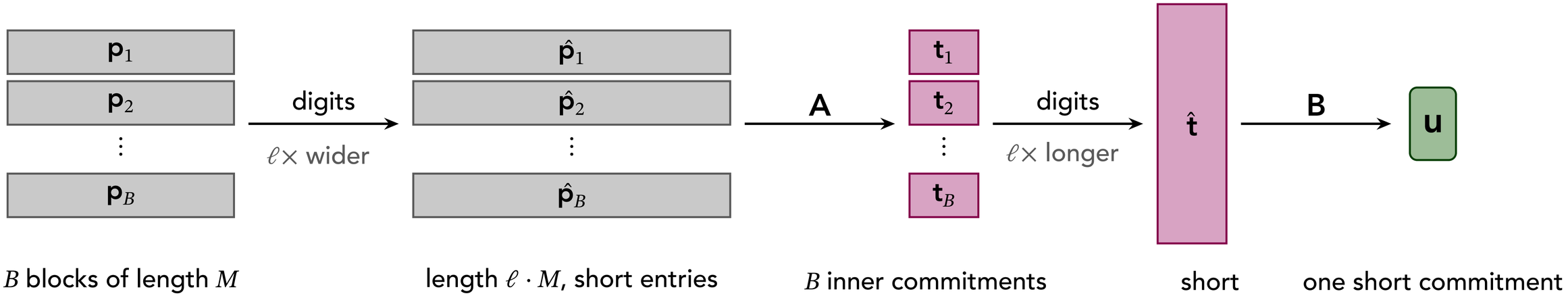
since $X^d = -1$

$\mathbf{a}$, $\mathbf{b}$, $\mathbf{o}$ are fixed by the point x

Two-Tier Commitments

Two-Tier Commitments

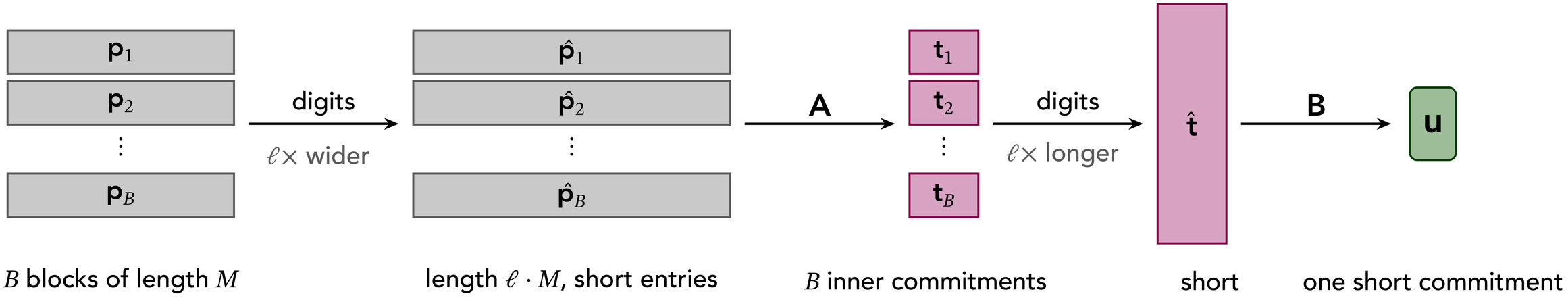
$\mathbf{u} = \mathrm{Commit}(p)$, computed in two tiers of Ajtai commitments



$\mathbf{A}$ and $\mathbf{B}$ are random matrices in the public parameters, can be chosen to be prefixes of a seed-derived setup

Two-Tier Commitments

$\mathbf{u} = \mathrm{Commit}(\boldsymbol{p})$, **computed in two tiers of Ajtai commitments**



- **Ajtai:** $\mathbf{A} \cdot \mathbf{s}$ binds only a short $\mathbf{s}$ (Module-SIS) $\Longrightarrow$ every input is first decomposed in small digits
- **Inner tier:** one matrix $\mathbf{A}$ for every block $\Longrightarrow$ a combination of blocks has the same combination of the $\mathbf{t}_i$
- **Outer tier:** the $\mathbf{t}_i$ are too many to send $\Longrightarrow$ $\mathbf{B}$ binds all of them in one short $\mathbf{u}$

$\mathbf{A}$ and $\mathbf{B}$ are random matrices in the public parameters, can be chosen to be prefixes of a seed-derived setup

Ring Relations and Norm Checks

Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$

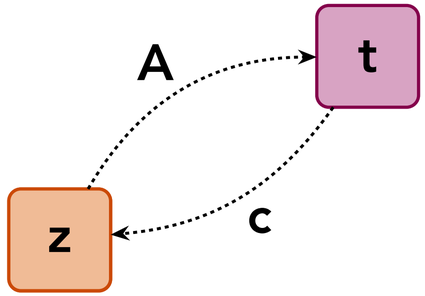
u

z

y

Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$



$\mathbf{u}$

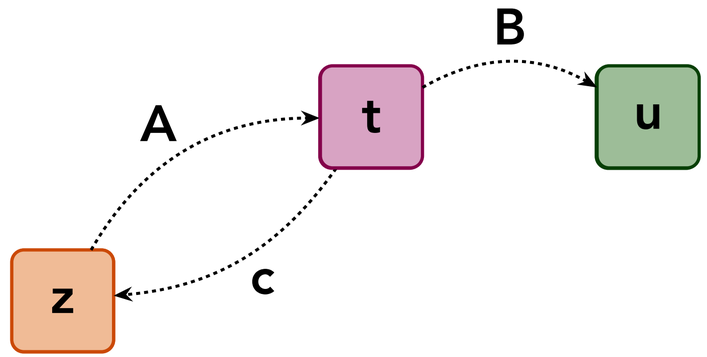
$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i \mathbf{t}_i$$

y

Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$



$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i \mathbf{t}_i \qquad\qquad \mathbf{u} = \mathbf{B}\,\hat{\mathbf{t}}$$

y

Ring Relations and Norm Checks

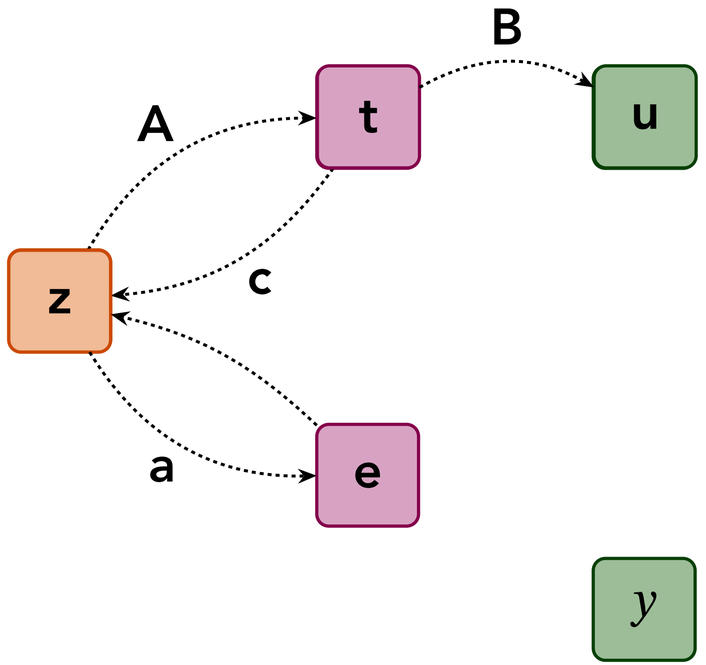
The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$

$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i \mathbf{t}_i \qquad\qquad \mathbf{u} = \mathbf{B}\,\hat{\mathbf{t}}$$

$\mathbf{z}$ agrees with the partial evaluations $\mathbf{e}_i = \langle \mathbf{p}_i, \mathbf{a} \rangle$:

$$\mathbf{a}^\top \mathbf{G}\,\mathbf{z} = \sum_i c_i \mathbf{e}_i$$



Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$

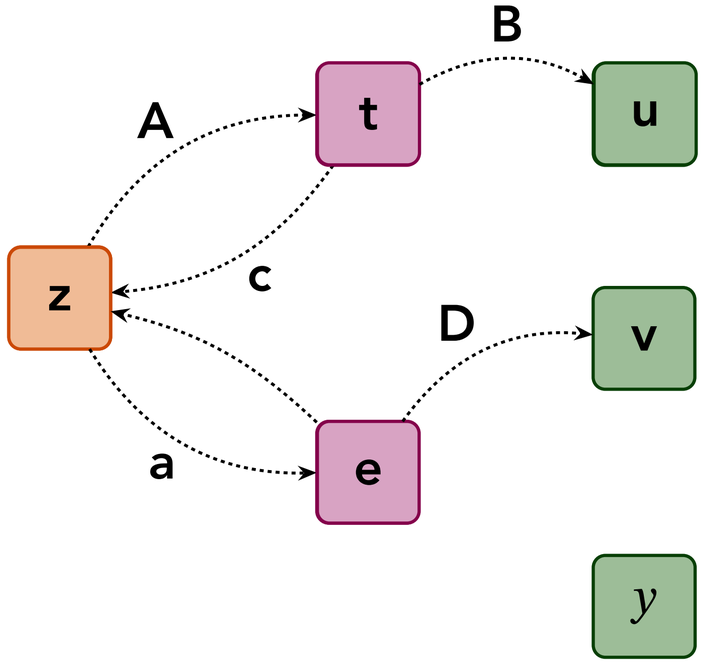
$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i \mathbf{t}_i \qquad\qquad \mathbf{u} = \mathbf{B}\,\hat{\mathbf{t}}$$

$\mathbf{z}$ agrees with the partial evaluations $\mathbf{e}_i = \langle \mathbf{p}_i, \mathbf{a} \rangle$:

$$\mathbf{a}^\top \mathbf{G}\,\mathbf{z} = \sum_i c_i \mathbf{e}_i \qquad\qquad \mathbf{v} = \mathbf{D}\,\hat{\mathbf{e}}$$

$\mathbf{v}$ is sent before the challenges c_i



Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$

$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i \mathbf{t}_i \qquad\qquad \mathbf{u} = \mathbf{B}\,\hat{\mathbf{t}}$$

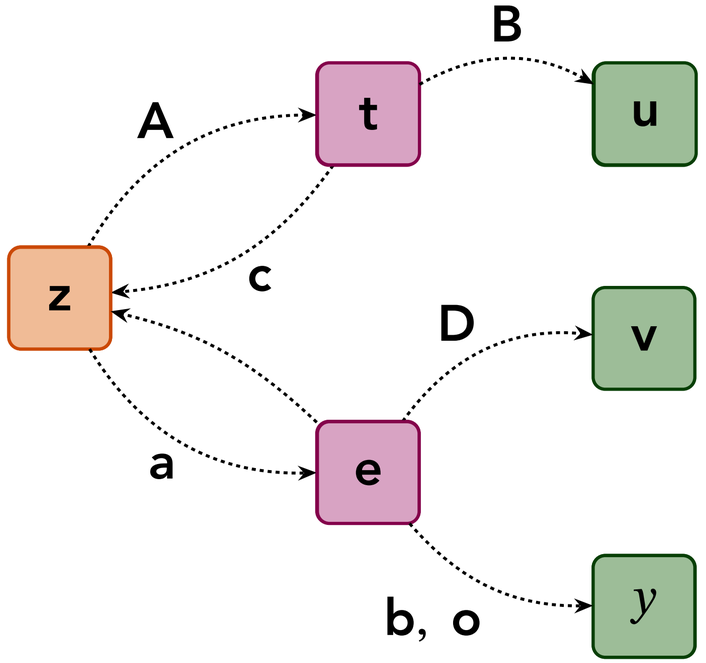
$\mathbf{z}$ agrees with the partial evaluations $\mathbf{e}_i = \langle \mathbf{p}_i, \mathbf{a} \rangle$:

$$\mathbf{a}^\top \mathbf{G}\,\mathbf{z} = \sum_i c_i \mathbf{e}_i \qquad\qquad \mathbf{v} = \mathbf{D}\,\hat{\mathbf{e}}$$

$\mathbf{v}$ is sent before the challenges c_i

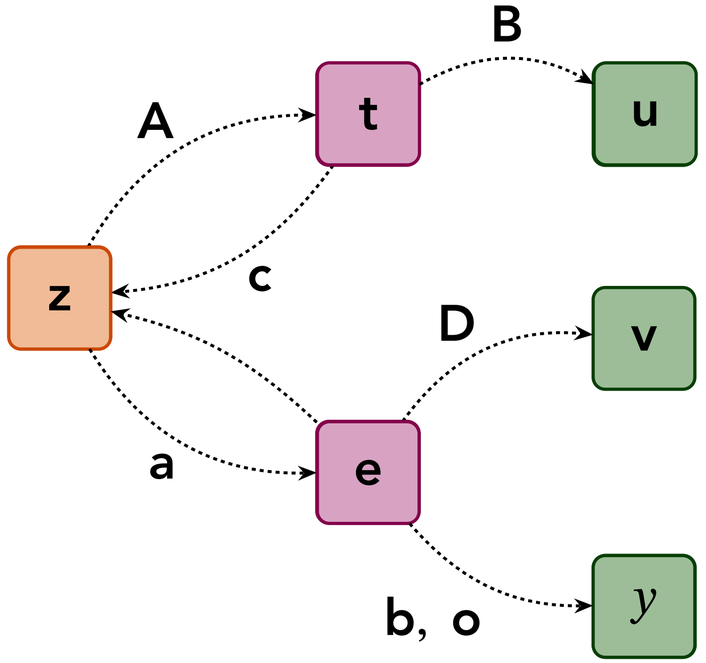
the partial evaluations give y:

$$y = \mathrm{ct}\Big(\sum_i \mathbf{b}_i \mathbf{e}_i \cdot \sigma_{-1}(\mathbf{o}) \Big)$$



Ring Relations and Norm Checks

The verifier has $\mathbf{u}, x, y$ and the challenges; $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$



$\mathbf{z}$ folds the committed blocks:

$$\mathbf{A}\,\mathbf{z} = \sum_i c_i\,\mathbf{t}_i \qquad\qquad \mathbf{u} = \mathbf{B}\,\hat{\mathbf{t}}$$

$\mathbf{z}$ agrees with the partial evaluations $\mathbf{e}_i = \langle \mathbf{p}_i, \mathbf{a} \rangle$:

$$\mathbf{a}^\top \mathbf{G}\,\mathbf{z} = \sum_i c_i\,\mathbf{e}_i \qquad\qquad \mathbf{v} = \mathbf{D}\,\hat{\mathbf{e}}$$

$\mathbf{v}$ is sent before the challenges c_i

the partial evaluations give y:

$$y = \mathrm{ct}\Big(\sum_i \mathbf{b}_i\,\mathbf{e}_i \cdot \sigma_{-1}(\mathbf{o}) \Big)$$

Alternatively: prove smallness in L_2 norm

Norm check: every digit of $\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}$ lies in $[-b/2, b/2)$

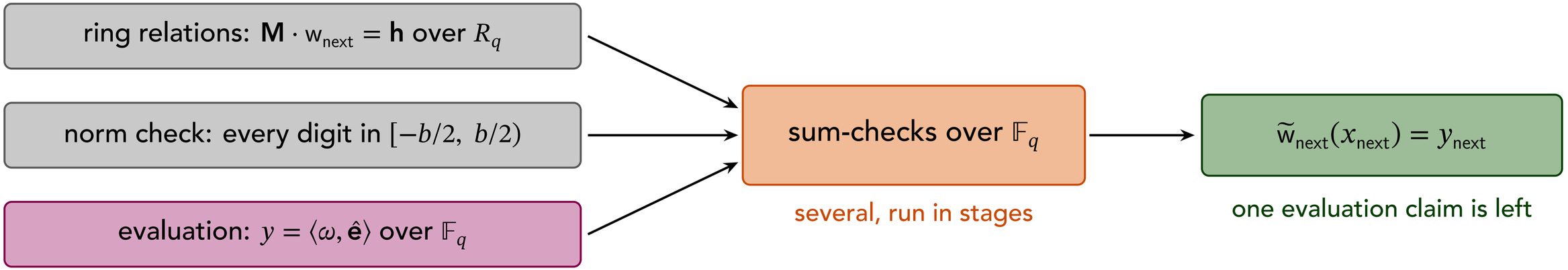
Needed in security proof, since commitments only bind short vectors under Module-SIS.

$\mathbf{z}$ grows in the fold, so it is decomposed again: $\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}$ enter the witness of the next fold.

Proving the Fold with Sum-Checks

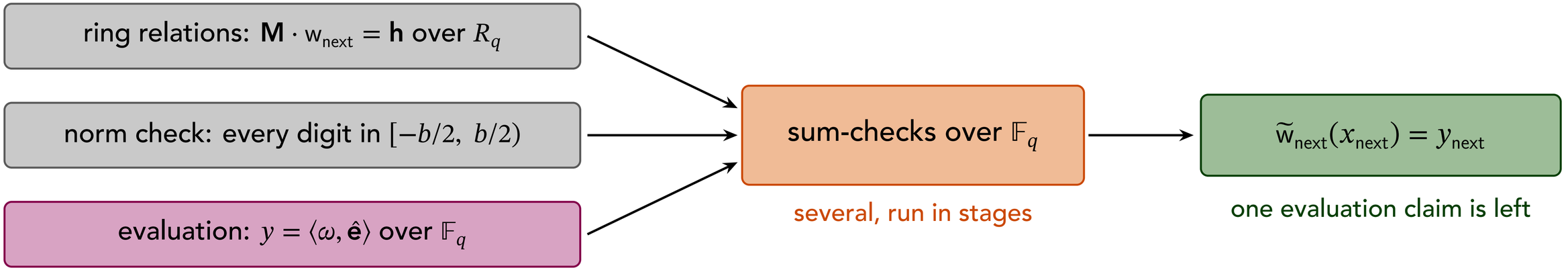
Proving the Fold with Sum-Checks

A sum-check-based PIOP for the committed witness $\mathsf{w}_{\text{next}} = (\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}, \hat{\mathbf{r}})$



Proving the Fold with Sum-Checks

A sum-check-based PIOP for the committed witness $\mathsf{w}_{\text{next}} = (\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}, \hat{\mathbf{r}})$

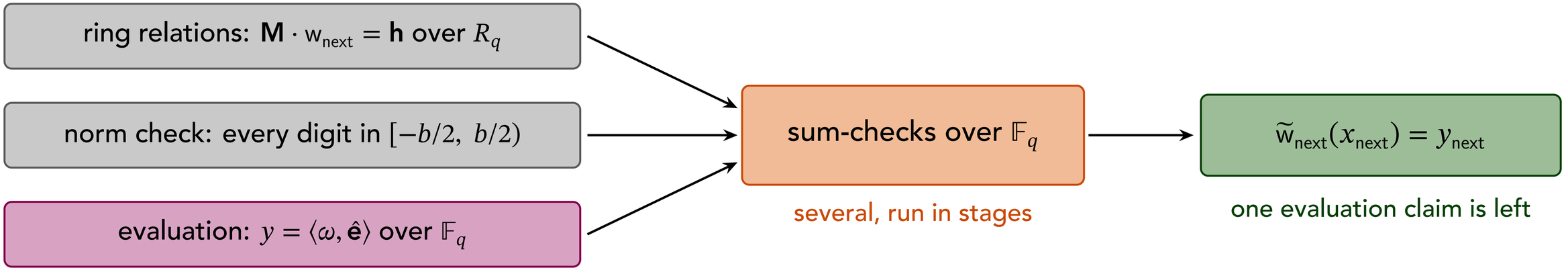


Ring switching [HMZ25, NOZ26]**: from ring relations to field relations**

- an equation modulo $X^d + 1$ is a polynomial identity with a quotient $\mathbf{r}$ $\Longrightarrow$ check it at a random $X = \alpha$
- the quotient joins the witness; every sum-check runs over $\mathbb{F}_q$, not over the ring

Proving the Fold with Sum-Checks

A sum-check-based PIOP for the committed witness $\mathsf{w}_{\text{next}} = (\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}, \hat{\mathbf{r}})$



Ring switching [HMZ25, NOZ26]: from ring relations to field relations

- an equation modulo $X^d + 1$ is a polynomial identity with a quotient $\mathbf{r}$ $\Longrightarrow$ check it at a random $X = \alpha$
- the quotient joins the witness; every sum-check runs over $\mathbb{F}_q$, not over the ring

Evaluation, proved on the digits (an Akita optimization)

- **Hachi** sends the ring element $\sum_i b_i \mathbf{e}_i$, about 1-4 KB per fold, and checks its constant term
- **Akita** sends no ring element: an inner-product sum-check proves $y = \sum_x \widetilde{\omega}(x) \cdot \widetilde{\hat{\mathbf{e}}}(x)$

One PCS Folding Step

One PCS Folding Step

PCS claim: $p(x) = y$, for the polynomial committed in $\mathbf{u}$

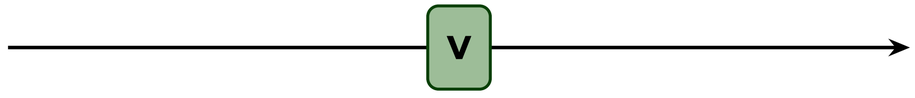
One PCS Folding Step

PCS claim: $p(x) = y$, **for the polynomial committed in** $\mathbf{u}$

Prover knows **p**

Verifier knows **u**, x, y

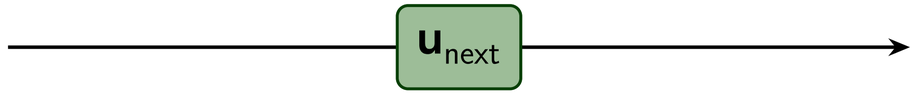
Evaluates each block: $\mathbf{e}_1, \ldots, \mathbf{e}_B$
Commits to their digits

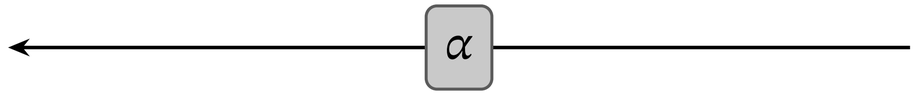




Samples short ring challenges

Folds the blocks: $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$
Computes the quotients **r**
Commits to $\mathsf{w}_{\mathsf{next}} = (\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}, \hat{\mathbf{r}})$





Samples a point α:
the relations are checked at $X = \alpha$

Proves: the relations hold at α,
every digit is in range,
and the evaluation y



Checks every round
Evaluates $\widetilde{m}(x_{\mathsf{next}})$

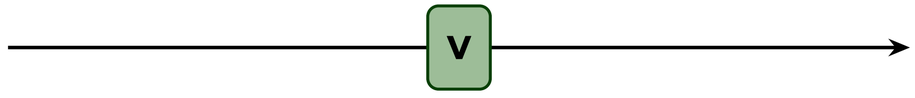
One PCS Folding Step

PCS claim: $p(x) = y$, **for the polynomial committed in** $\mathbf{u}$

Prover knows **p**

Verifier knows **u**, x, y

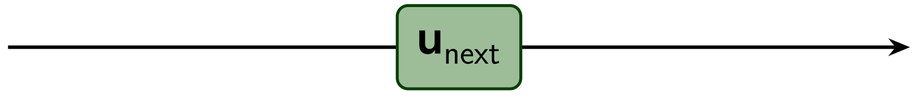
Evaluates each block: $\mathbf{e}_1, \ldots, \mathbf{e}_B$
Commits to their digits

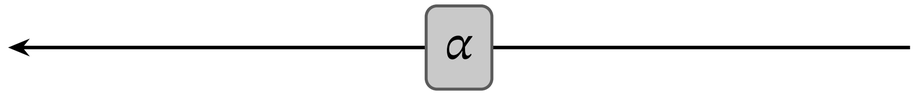




Samples short ring challenges

Folds the blocks: $\mathbf{z} = \sum_i c_i \hat{\mathbf{p}}_i$
Computes the quotients **r**
Commits to $\mathsf{w}_{\mathsf{next}} = (\hat{\mathbf{z}}, \hat{\mathbf{t}}, \hat{\mathbf{e}}, \hat{\mathbf{r}})$





Samples a point α:
the relations are checked at $X = \alpha$

Proves: the relations hold at α,
every digit is in range,
and the evaluation y



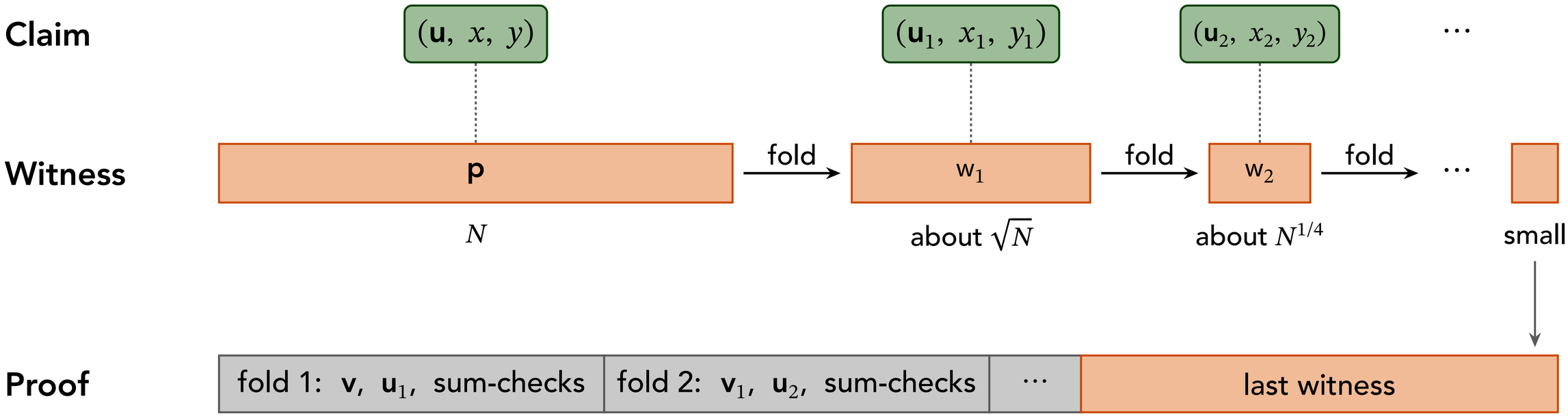
Checks every round
Evaluates $\widetilde{m}(x_{\mathsf{next}})$

What is left: $\widetilde{\mathsf{w}}_{\mathsf{next}}(x_{\mathsf{next}}) = y_{\mathsf{next}}$, **for the witness committed in** $\mathbf{u}_{\mathsf{next}}$

The Whole PCS Protocol

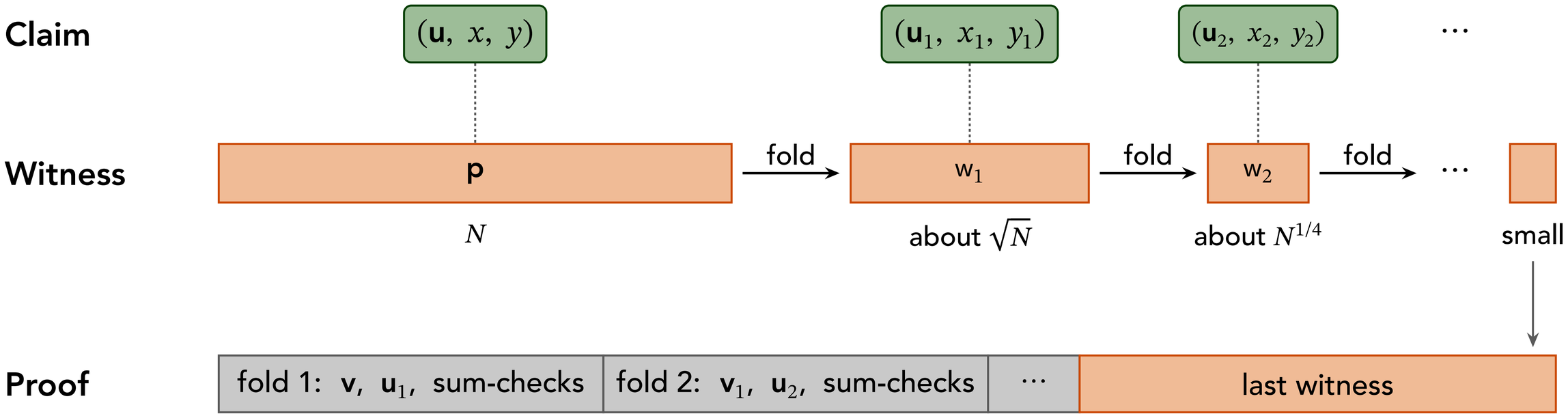
The Whole PCS Protocol

We fold until the witness is small, then send it in full



The Whole PCS Protocol

We fold until the witness is small, then send it in full



- **Each fold:** B blocks of length N/B leave a witness of about $N/B + B$, so $B = \sqrt{N}$
- **The proof stays short:** each fold adds two commitments and its sum-check messages
- **The first fold is the largest:** main contributor to prover and verifier time

Roadmap

1. The Lattice-Based PCS Template
2. Akita and Its Core Techniques
3. Evaluation & Future Work

PCS Fold: Bottlenecks and Improvements

PCS Fold: Bottlenecks and Improvements

Akita introduces new techniques to handle each downside of the folding template:

PCS Fold: Bottlenecks and Improvements

Akita introduces new techniques to handle each downside of the folding template:

Verifier time

the verifier evaluates the commitment matrices itself:
about $B + N/B$ ring operations, at best $\sqrt{N}$



Technique 1:
Setup Offloading

PCS Fold: Bottlenecks and Improvements

Akita introduces new techniques to handle each downside of the folding template:

Verifier time

the verifier evaluates the commitment matrices itself:
about $B + N/B$ ring operations, at best $\sqrt{N}$

→ **Technique 1:**
Setup Offloading

Norm check

digits in base b requires a single sum-check of degree $b + 1$

→ **Technique 2:**
(Multi-Stage)
Digit Range Check

PCS Fold: Bottlenecks and Improvements

Akita introduces new techniques to handle each downside of the folding template:

Verifier time

the verifier evaluates the commitment matrices itself:
about $B + N/B$ ring operations, at best $\sqrt{N}$

→ **Technique 1:**
Setup Offloading

Norm check

digits in base b requires a single sum-check of degree $b + 1$

→ **Technique 2:**
(Multi-Stage)
Digit Range Check

Commitment size

every fold sends two commitments of several kilobytes

→ **Technique 3:**
Compressed Commitments

… and more!

The Verifier's Work in One Fold

The Verifier's Work in One Fold

The last sum-check of a fold proves a sum over the next witness: m: the rows of the matrix, batched

$$\sum_x \widetilde{w}_{\text{next}}(x) \cdot \widetilde{m}(x) = \text{claimed value}$$

$$\widetilde{w}_{\text{next}}(x_{\text{next}}) \cdot \widetilde{m}(x_{\text{next}}) = \text{last sum-check message}$$

the claim for the next fold

the verifier computes this itself

The Verifier's Work in One Fold

The last sum-check of a fold proves a sum over the next witness: m: the rows of the matrix, batched

$$\sum_x \widetilde{w}_{\text{next}}(x) \cdot \widetilde{m}(x) = \text{claimed value}$$

$$\widetilde{w}_{\text{next}}(x_{\text{next}}) \cdot \widetilde{m}(x_{\text{next}}) = \text{last sum-check message}$$

the claim for the next fold

the verifier computes this itself

Folding requires generating challenges $c_1, \ldots, c_B$

Other $O(\log N)$ sum-check rounds are cheap.

The Verifier's Work in One Fold

The last sum-check of a fold proves a sum over the next witness: m: the rows of the matrix, batched

$$\sum_x \widetilde{w}_{\text{next}}(x) \cdot \widetilde{m}(x) = \text{claimed value}$$

$$\widetilde{w}_{\text{next}}(x_{\text{next}}) \cdot \widetilde{m}(x_{\text{next}}) = \text{last sum-check message}$$

the claim for the next fold

the verifier computes this itself

Folding requires generating challenges $c_1, \ldots, c_B$

Other $O(\log N)$ sum-check rounds are cheap.

$\Longrightarrow$ **the verifier needs to balance the costs: evaluating $\widetilde{m}(x_{\text{next}})$ & processing ring challenges**

Technique 1: Setup Offloading

Technique 1: Setup Offloading

Where the time goes: $\widetilde{m}(x_{\text{next}})$, for a witness of B blocks of length N/B

$\hat{\mathbf{z}}$	$\hat{\mathbf{t}}$	$\hat{\mathbf{e}}$	$\hat{\mathbf{r}}$
0	$\mathbf{B}_\alpha$	0	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
0	0	$\mathbf{D}_\alpha$	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
$(\mathbf{AG})_\alpha$	$(-\mathbf{c}^\top \otimes \mathbf{G})_\alpha$	0	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
$(-\mathbf{a}^\top \mathbf{GG})_\alpha$	0	$(\mathbf{c}^\top \otimes \mathbf{G})_\alpha$	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
0	0	ω_{eval}	0

structured: gadgets, evaluation points	$O(\log N)$
folding challenges $c_1, \ldots, c_B$	$O(B)$
setup matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{D}$: random	$O(N/B + B)$

Technique 1: Setup Offloading

Where the time goes: $\widetilde{m}(x_{\text{next}})$, for a witness of B blocks of length N/B

$\hat{\mathbf{z}}$	$\hat{\mathbf{t}}$	$\hat{\mathbf{e}}$	$\hat{\mathbf{r}}$
0	$\mathbf{B}_\alpha$	0	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
0	0	$\mathbf{D}_\alpha$	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
$(\mathbf{AG})_\alpha$	$(-\mathbf{c}^\top \otimes \mathbf{G})_\alpha$	0	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
$(-\mathbf{a}^\top \mathbf{GG})_\alpha$	0	$(\mathbf{c}^\top \otimes \mathbf{G})_\alpha$	$-(\alpha^d+1)\,\mathbf{G}_\alpha$
0	0	ω_{eval}	0

structured: gadgets, evaluation points	$O(\log N)$
folding challenges $c_1, \ldots, c_B$	$O(B)$
setup matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{D}$: random	$O(N/B + B)$

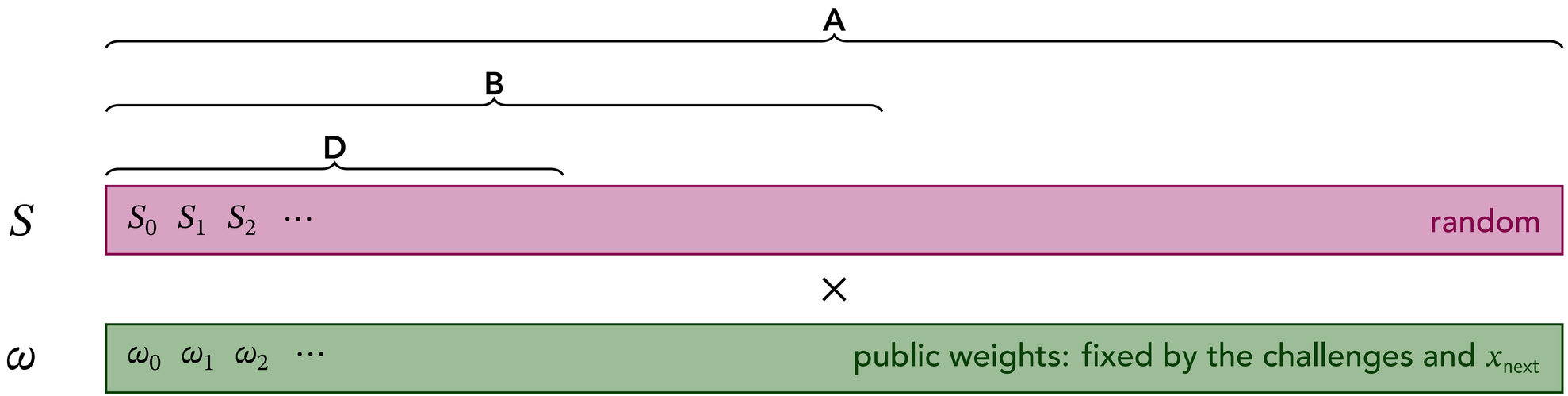
Total: $B + N/B \geq 2\sqrt{N}$. Fewer blocks mean fewer challenges, but a wider $\mathbf{A}$.

Polylog verification is possible with structured matrices, but relies on the vanishing-SIS assumption.

Technique 1: Setup Offloading

Technique 1: Setup Offloading

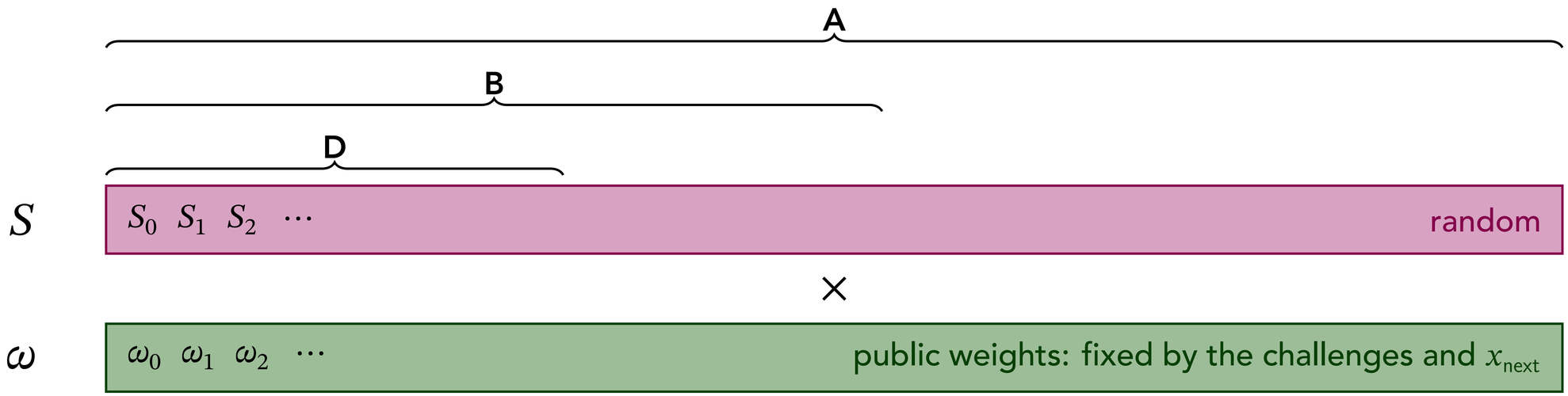
$\mathbf{A}$, $\mathbf{B}$, $\mathbf{D}$ are prefixes of one random sequence S, expanded from a seed



$\tilde{m}(x_{\text{next}})$ = structured + challenges + σ where $\sigma = \sum_p S_p \cdot \omega_p$

Technique 1: Setup Offloading

$\mathbf{A}$, $\mathbf{B}$, $\mathbf{D}$ are prefixes of one random sequence S, expanded from a seed



$\widetilde{m}(x_{\text{next}})$ = structured + challenges + σ where $\sigma = \sum_p S_p \cdot \omega_p$

- The verifier computes the structured and challenge terms itself: $O(\log N + B)$
- The prover sends σ, the setup term; it remains to prove that σ is correct

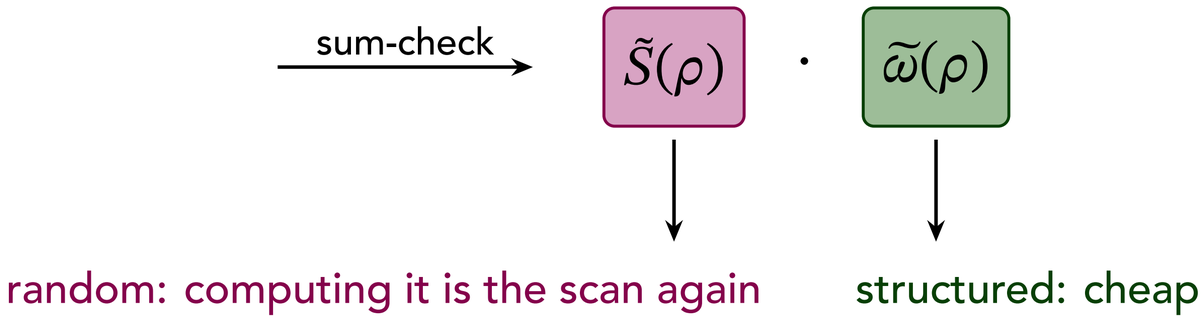
Technique 1: Setup Offloading

Technique 1: Setup Offloading

A sum-check reduces σ to one evaluation of S

View S and ω as multilinear polynomials:

$$\sigma = \sum_{p \in \{0,1\}^{\nu}} \tilde{S}(p) \cdot \tilde{\omega}(p)$$

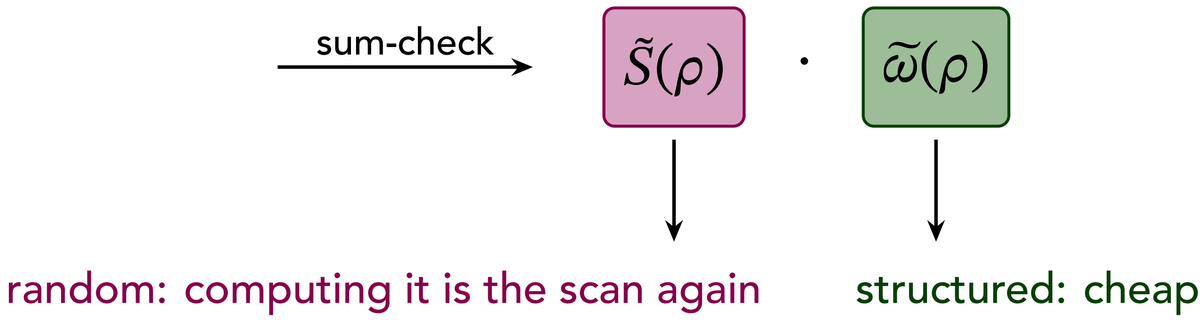


Technique 1: Setup Offloading

A sum-check reduces σ to one evaluation of S

View S and ω as multilinear polynomials:

$$\sigma = \sum_{p \in \{0,1\}^{\nu}} \tilde{S}(p) \cdot \tilde{\omega}(p)$$



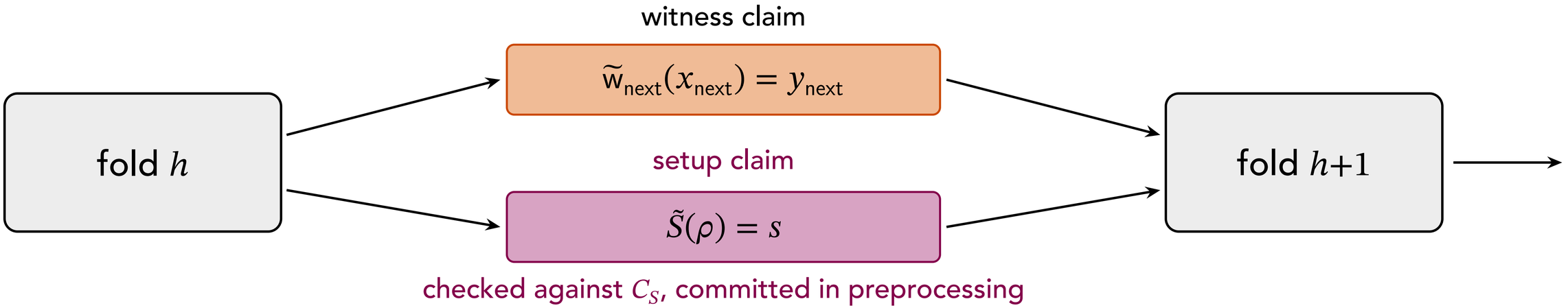
But S is fixed at setup: commit to it once, in preprocessing: $C_S = \text{Commit}(S)$

$\tilde{S}(\rho) = s$ is an evaluation claim for our own PCS

Technique 1: Setup Offloading

Technique 1: Setup Offloading

The setup claim joins the next fold



- The next fold opens w_{next} and S together, batched by a random challenge
- S is around the size of w_{next}: the next fold grows by a constant factor
- The next fold(s) can offload again, until it is no longer profitable to offload

Technique 1: Setup Offloading

Technique 1: Setup Offloading

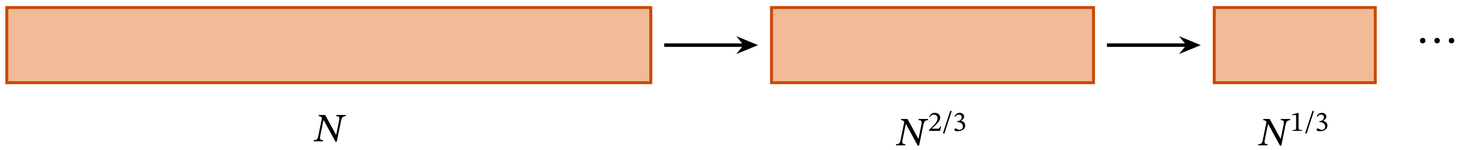
Wide folds: $B = N^{1/K}$ blocks in each of the first $K-1$ folds

Template
$B = \sqrt{N}$



verifier: $\sqrt{N}$

Akita, $K = 3$
$B = N^{1/3}$



verifier: $N^{1/3}$

Technique 1: Setup Offloading

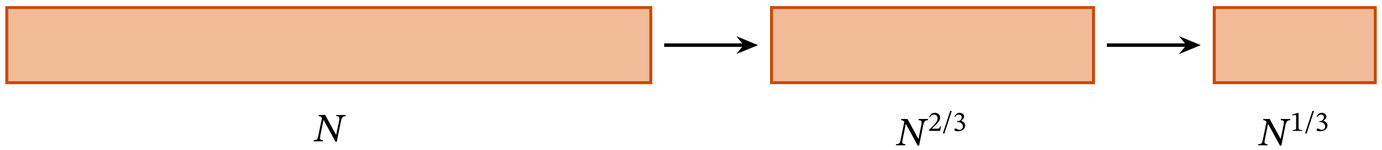
Wide folds: $B = N^{1/K}$ blocks in each of the first $K - 1$ folds

Template
$B = \sqrt{N}$



verifier: $\sqrt{N}$

Akita, $K = 3$
$B = N^{1/3}$



verifier: $N^{1/3}$

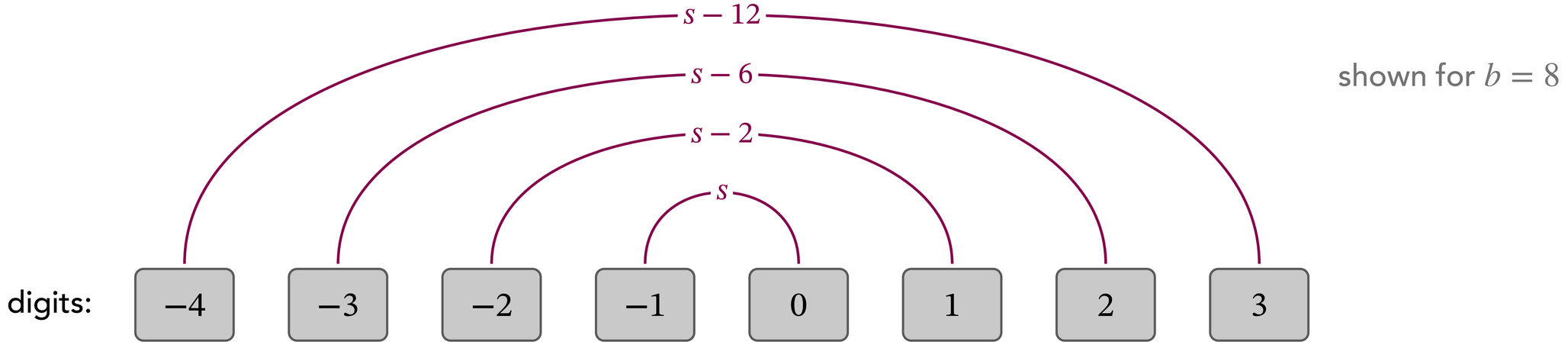
- Verifier time $\widetilde{O}(N^{1/K})$ for any constant K; prover $\widetilde{O}(N)$ and proof size as before
- At 4 GiB over 32-bit field: verification 32 ms to 14 ms; opening 5.7 s to 8.4 s, proof +2.7 KB
- A planner offloads only in the folds where it pays

Technique 2: Digit Range Check

Technique 2: Digit Range Check

Every digit of the witness must lie in $\{-b/2,\ldots,b/2-1\}$

Template: $\prod_{a=-b/2}^{b/2-1}(w-a)=0$ at every digit w: an equality-weighted sum-check of degree $b+1$

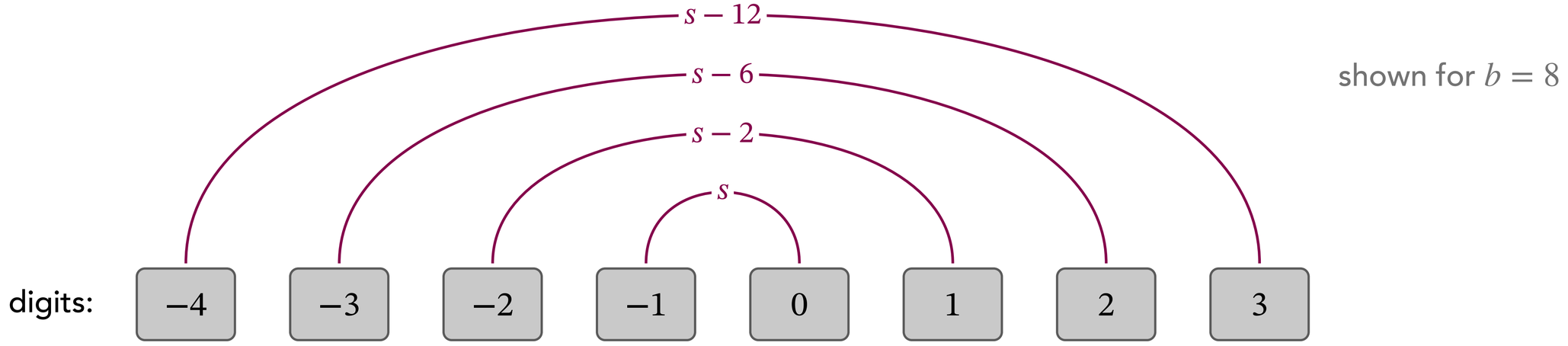


shown for $b=8$

Technique 2: Digit Range Check

Every digit of the witness must lie in $\{-b/2,\ldots,b/2-1\}$

Template: $\prod_{a=-b/2}^{b/2-1}(w-a)=0$ at every digit w: an equality-weighted sum-check of degree $b+1$



shown for $b=8$

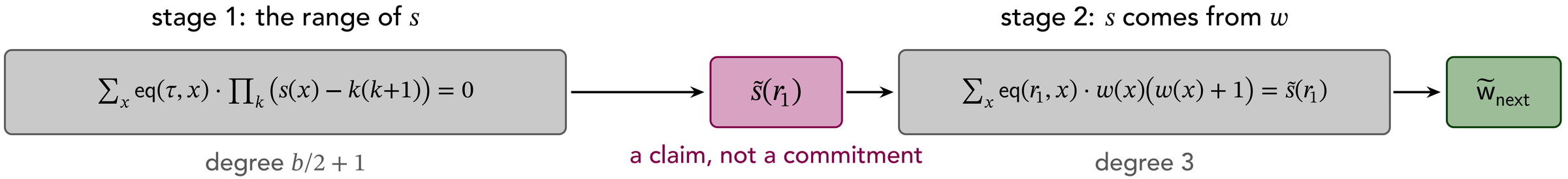
Pair the digit k with $-(k+1)$: $(w-k)(w+k+1)=s-k(k+1)$, where $s=w(w+1)$

The check becomes $\prod_{k=0}^{b/2-1}\big(s-k(k+1)\big)=0$: half the degree, in the variable s

Technique 2: Digit Range Check

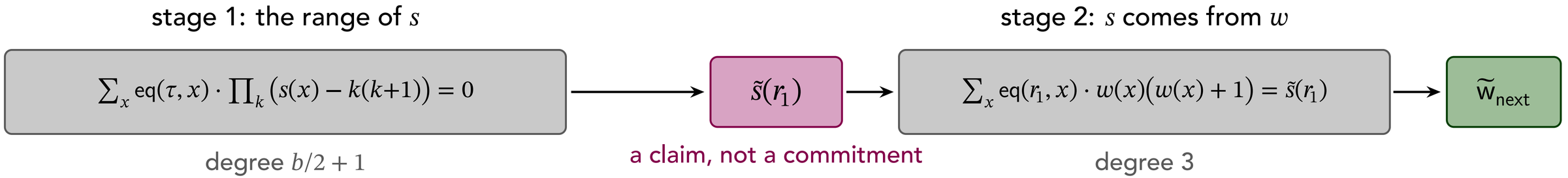
Technique 2: Digit Range Check

Two or more sum-checks of low degree in place of one of high degree



Technique 2: Digit Range Check

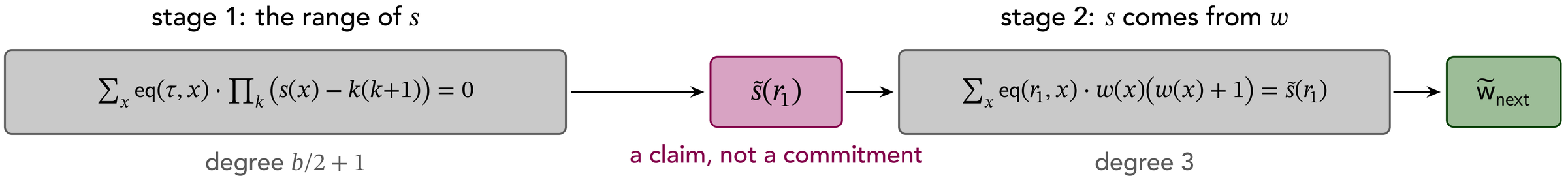
Two or more sum-checks of low degree in place of one of high degree



- s is never committed: stage 2 reduces its claim to the witness, inside the relation sum-check of the fold
- Large bases, $b \geq 16$: stage 1 splits again, as a product tree with two or four children per node
- Prover: w and s take few, small values, so most multiplications are cheap

Technique 2: Digit Range Check

Two or more sum-checks of low degree in place of one of high degree



- s is never committed: stage 2 reduces its claim to the witness, inside the relation sum-check of the fold
- Large bases, $b \geq 16$: stage 1 splits again, as a product tree with two or four children per node
- Prover: w and s take few, small values, so most multiplications are cheap

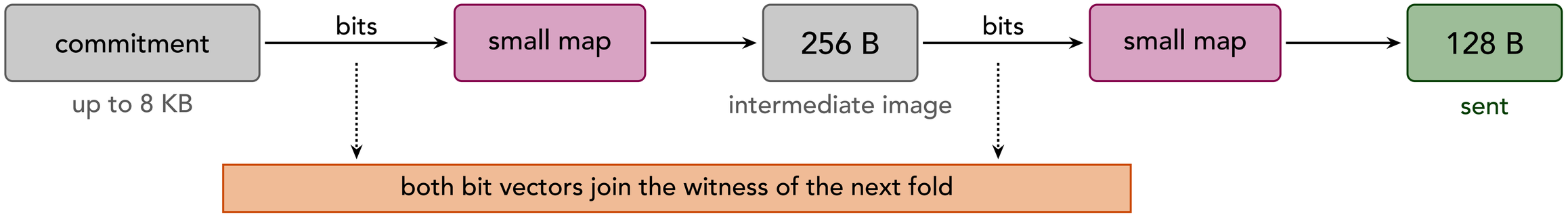
Sum-check messages of the range check, 2^{20} digits, baseline vs. optimized:

$b = 16$: 5.1 KB to 2.2 KB **$b = 32$: 10.2 KB to 2.9 KB**

Technique 3: Compressed Commitments

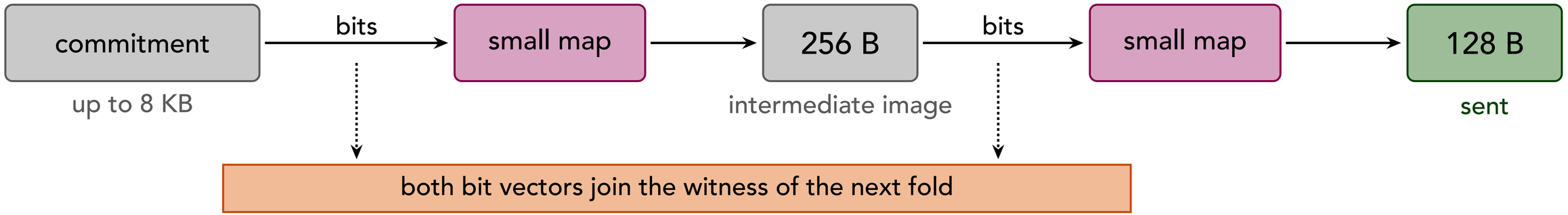
Technique 3: Compressed Commitments

Each fold sends two commitments, $\mathbf{v}$ and $\mathbf{u}_{\text{next}}$: usually 1 KB, up to 8 KB



Technique 3: Compressed Commitments

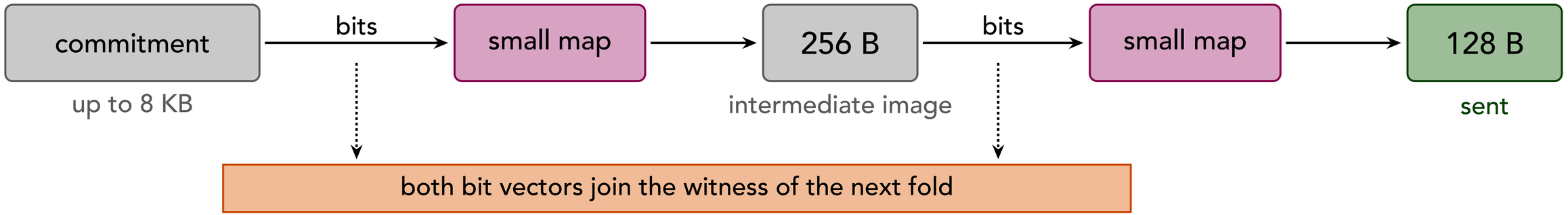
Each fold sends two commitments, $\mathbf{v}$ and $\mathbf{u}_{\text{next}}$: usually 1 KB, up to 8 KB



- **Small output:** bits give the smallest collision norm, so Module-SIS allows the shortest output
- **Two-stage compression:** SIS security depends on the input width, so a wide commitment is absorbed gradually
- Bits in $\{-1,0\}$ satisfy $w(w+1)=0 \Longrightarrow$ fused into the range check of Technique 2
- This adds to the next witness $\Longrightarrow$ compression is used in the early folds, where the witness is large

Technique 3: Compressed Commitments

Each fold sends two commitments, $\mathbf{v}$ and $\mathbf{u}_{\text{next}}$: usually 1 KB, up to 8 KB



- **Small output:** bits give the smallest collision norm, so Module-SIS allows the shortest output
- **Two-stage compression:** SIS security depends on the input width, so a wide commitment is absorbed gradually
- Bits in $\{-1,0\}$ satisfy $w(w+1)=0 \Longrightarrow$ fused into the range check of Technique 2
- This adds to the next witness $\Longrightarrow$ compression is used in the early folds, where the witness is large

Every compressed commitment is 128 bytes, the polynomial commitment included.

Orthus [BBLMS26] uses the same idea with a single compression step: its commitments are more than twice as large.

Roadmap

1. The Lattice-Based PCS Template
2. Akita and Its Core Techniques
3. Evaluation & Future Work

Evaluation: Setting & Methodology

Evaluation: Setting & Methodology

How does Akita compare with other post-quantum PCSs?

Evaluation: Setting & Methodology

How does Akita compare with other post-quantum PCSs?

How much better is Jolt with Akita?

Evaluation: Setting & Methodology

How does Akita compare with other post-quantum PCSs?

How much better is Jolt with Akita?

Each scheme has its own field. For a fair comparison, all commit the same amount of data, measured in bits:

$$\text{bits committed} = \text{field elements} \times \text{bits per element}$$

Evaluation: Setting & Methodology

How does Akita compare with other post-quantum PCSs?

How much better is Jolt with Akita?

Each scheme has its own field. For a fair comparison, all commit the same amount of data, measured in bits:

bits committed = field elements × bits per element

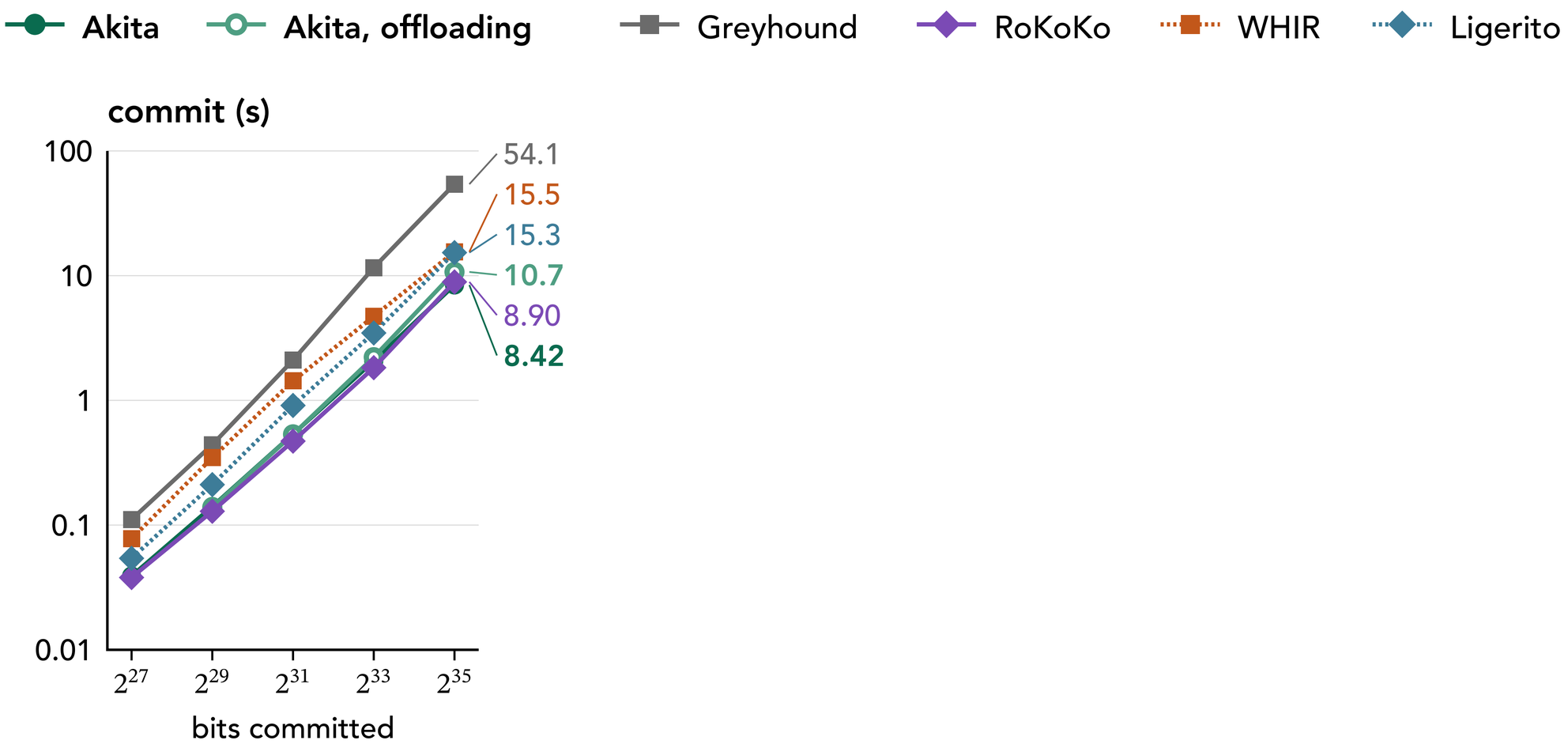
scheme	implementation	security	field order	elements at 2^{35} bits
Akita	ours, in Rust	128-bit, Module-SIS	$2^{32} - 99$	2^{30}
Greyhound	reference code, plus our fixes	128-bit, Module-SIS *	$2^{32} - 99$	2^{30}
RoKoKo	authors' code and parameters	100-bit, vanishing SIS	$2^{50} - 2687$	2^{30}, 31-bit values
WHIR	Plonky3, rate $1/2$	128-bit, Johnson bound	$2^{31} - 2^{24} + 1$	2^{30}
Ligerito	Flock, default fast profile	128-bit, Johnson bound	2^{128}	2^{28}

* The implementation we test uses an optimization that requires a non-standard assumption, truncated Ring-SIS, not pure Module-SIS.

Evaluation: PCS Prover

Evaluation: PCS Prover

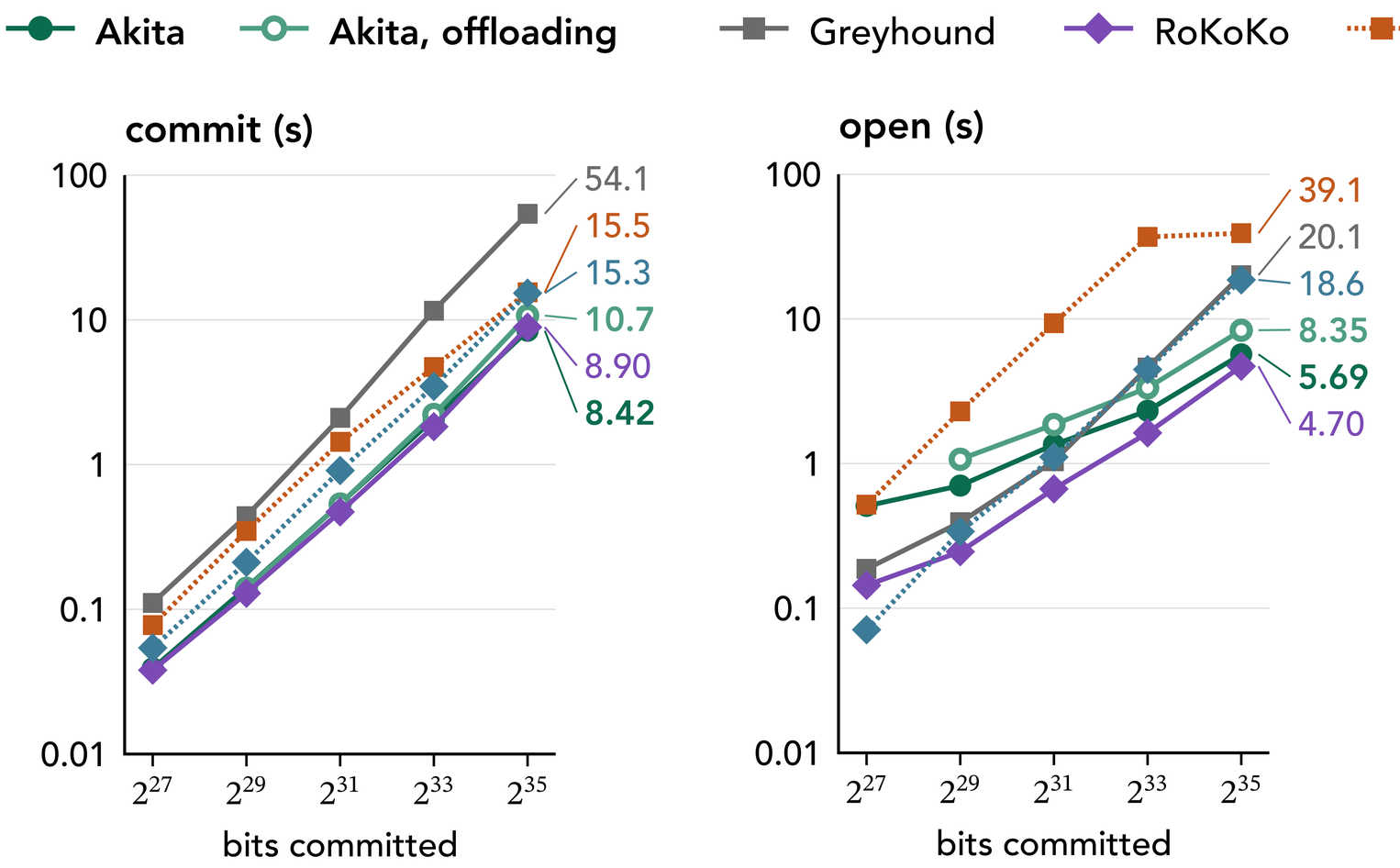
One thread (AMD Ryzen 9 9950X, AVX-512); value at 2^{35} bits next to each line



Median of ten runs. Commit and open are timed separately.

Evaluation: PCS Prover

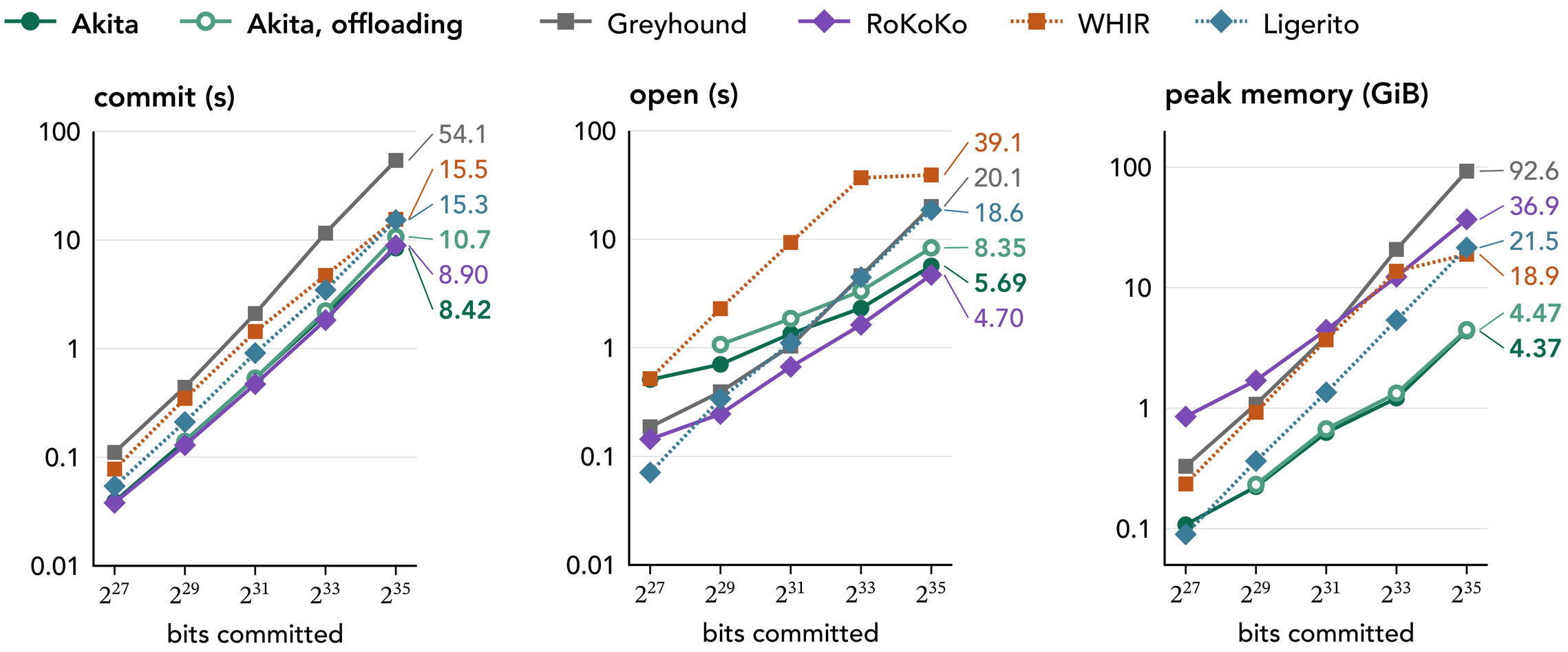
One thread (AMD Ryzen 9 9950X, AVX-512); value at 2^{35} bits next to each line



Median of ten runs. Commit and open are timed separately.

Evaluation: PCS Prover

One thread (AMD Ryzen 9 9950X, AVX-512); value at 2^{35} bits next to each line

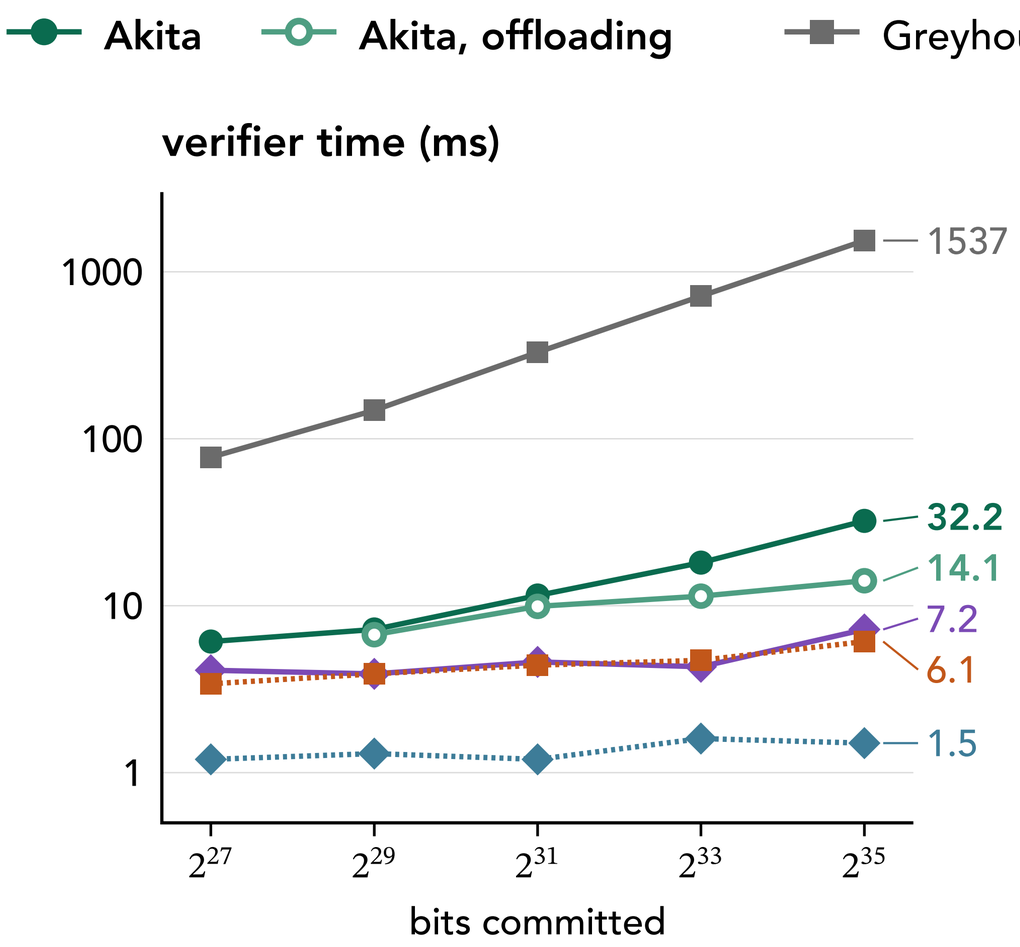


Median of ten runs. Commit and open are timed separately.

Evaluation: PCS Verifier and Proof Size

Evaluation: PCS Verifier and Proof Size

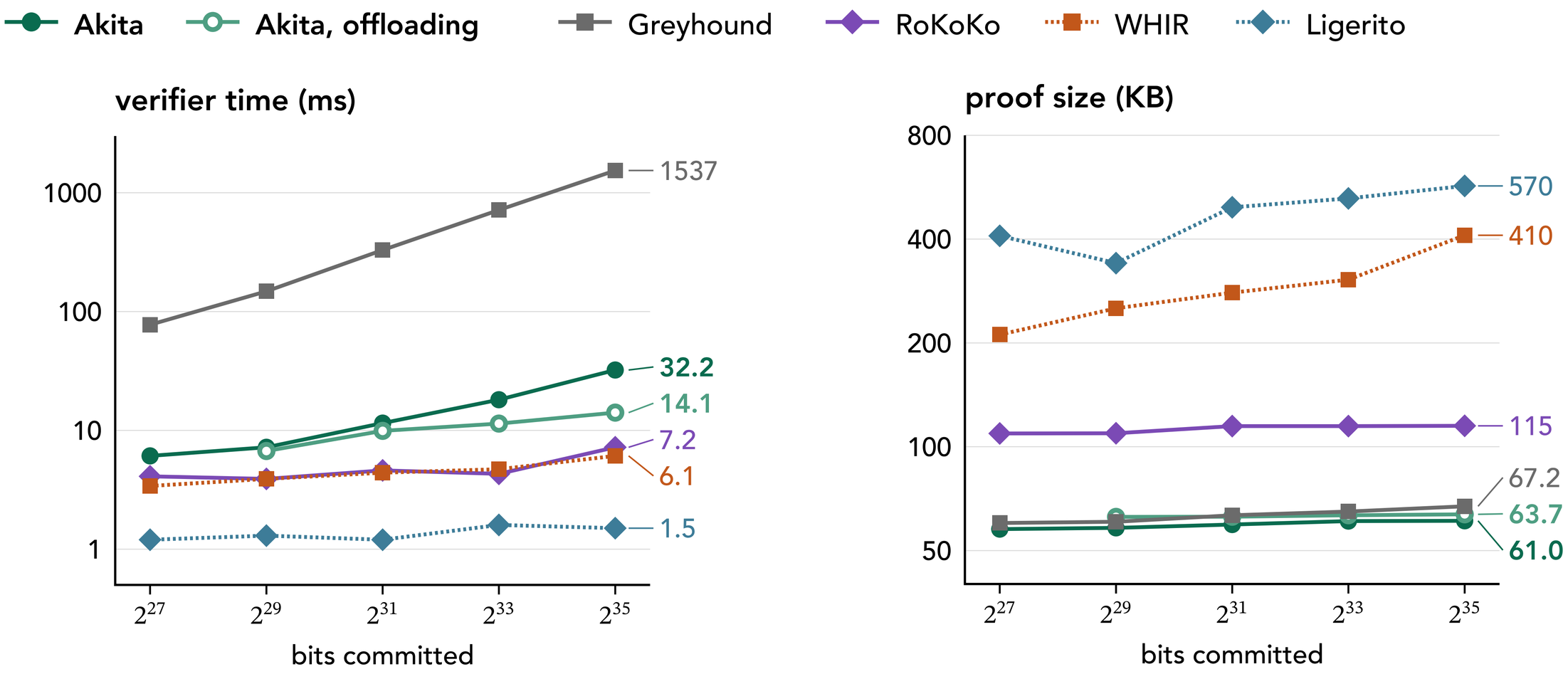
One thread (AMD Ryzen 9 9950X, AVX-512); value at 2^{35} bits next to each line



Median of ten runs. Proof size excludes the commitment.

Evaluation: PCS Verifier and Proof Size

One thread (AMD Ryzen 9 9950X, AVX-512); value at 2^{35} bits next to each line

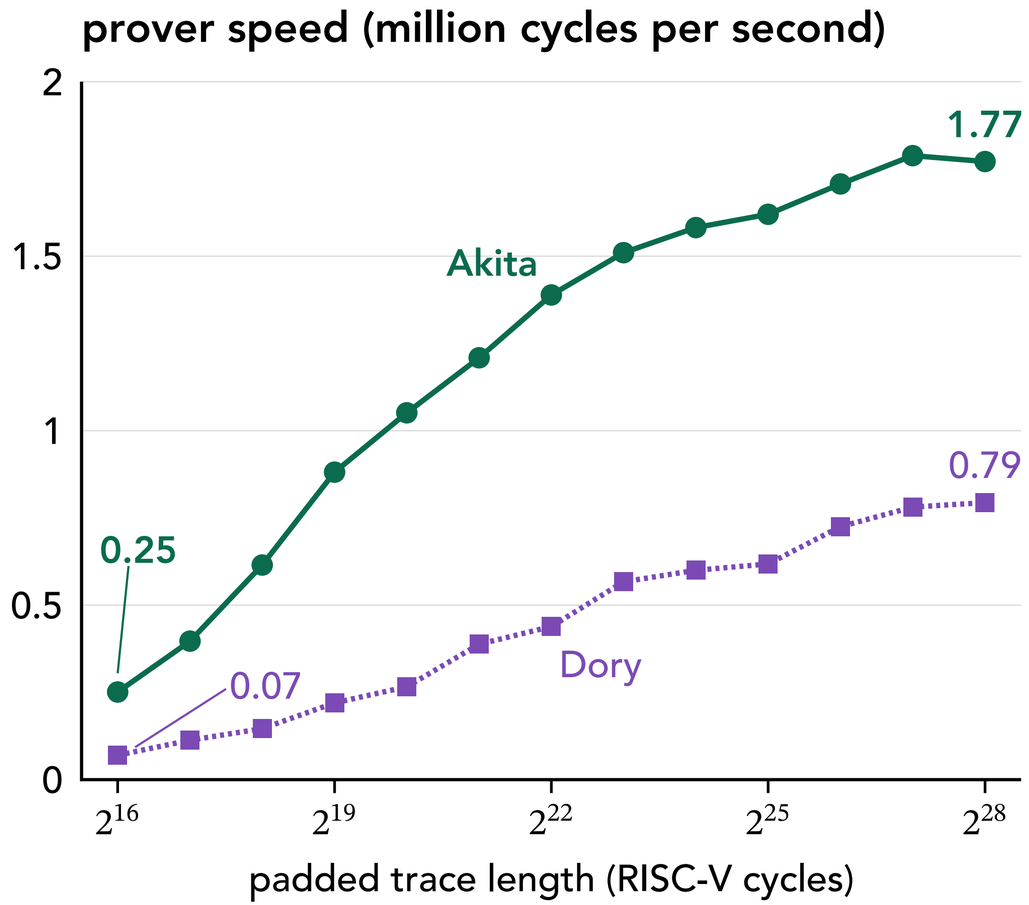


Median of ten runs. Proof size excludes the commitment.

Evaluation: Jolt Prover

Evaluation: Jolt Prover

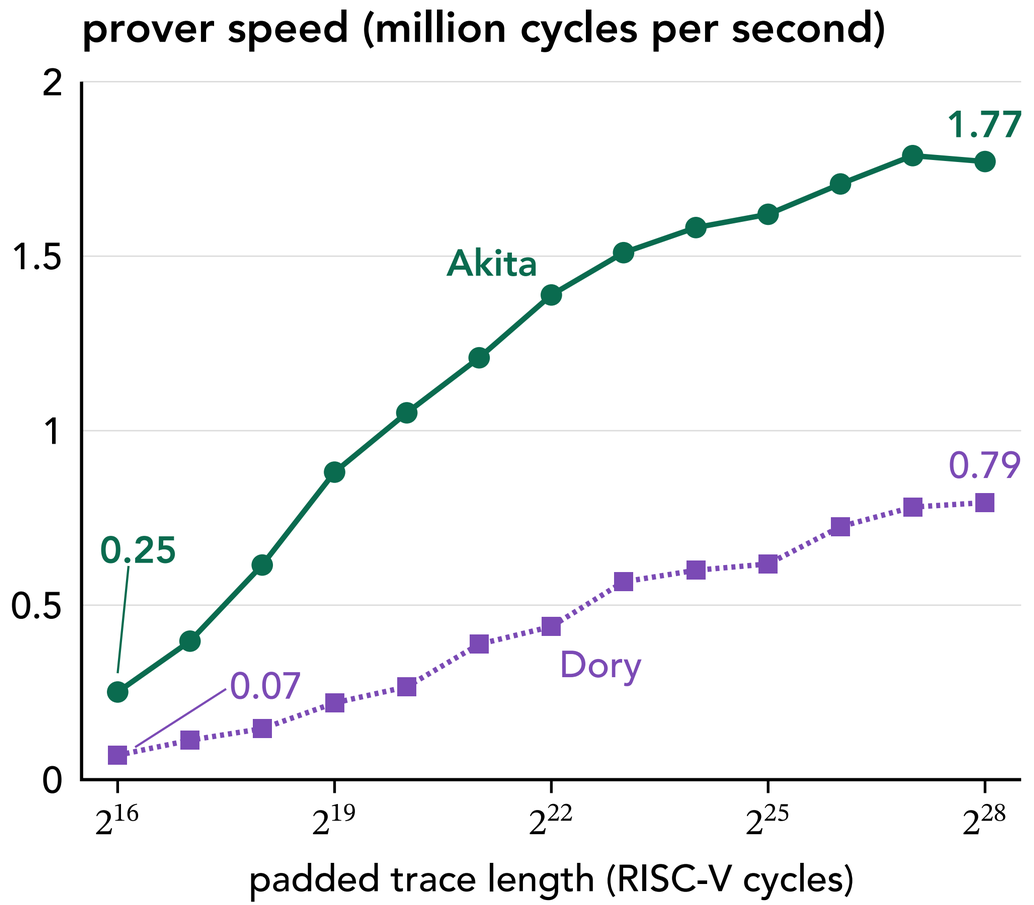
Jolt with Akita against Jolt with Dory; SHA-256 chains, Apple M4 Max (16 cores)



Median of three runs. Proving time excludes program preprocessing, PCS setup, and execution tracing.

Evaluation: Jolt Prover

Jolt with Akita against Jolt with Dory; SHA-256 chains, Apple M4 Max (16 cores)



proving time

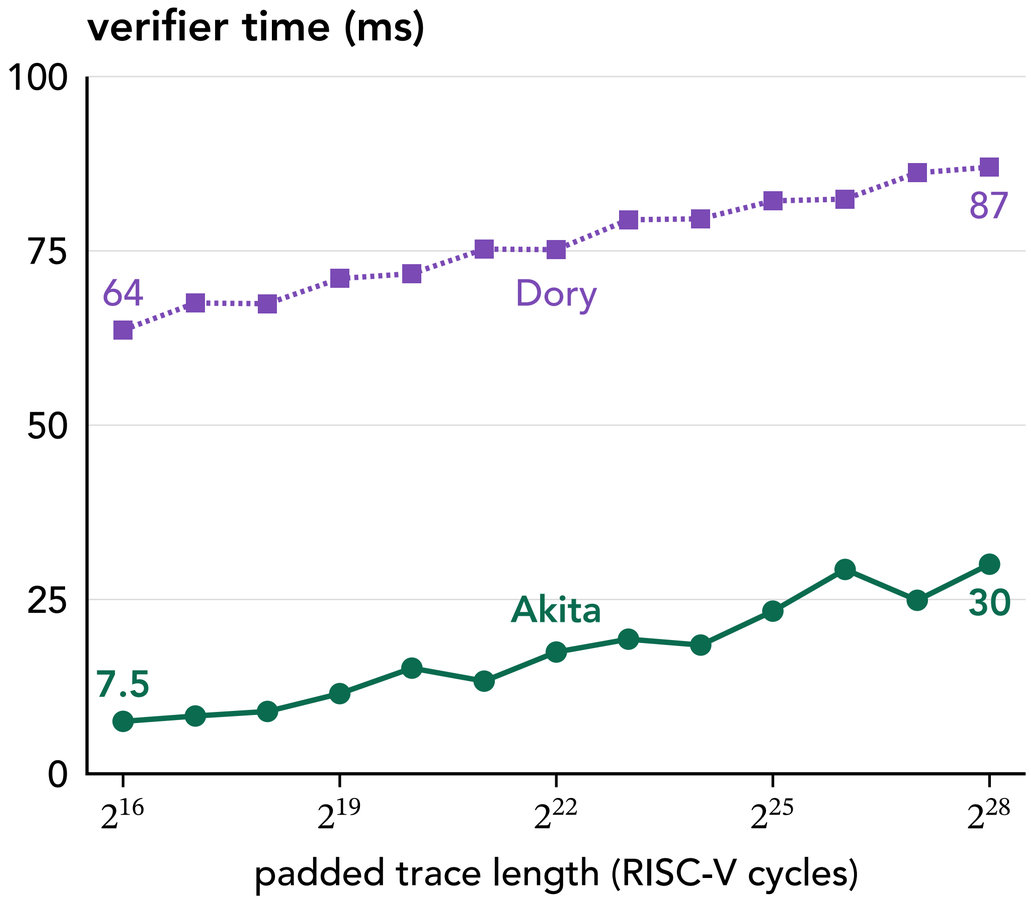
cycles	Akita	Dory	speedup
2^{16}	0.26 s	0.93 s	3.6×
2^{19}	0.59 s	2.38 s	4.0×
2^{22}	3.02 s	9.56 s	3.2×
2^{25}	20.7 s	54.3 s	2.6×
2^{28}	152 s	338 s	2.2×

Median of three runs. Proving time excludes program preprocessing, PCS setup, and execution tracing.

Evaluation: Jolt Verifier and Proof Size

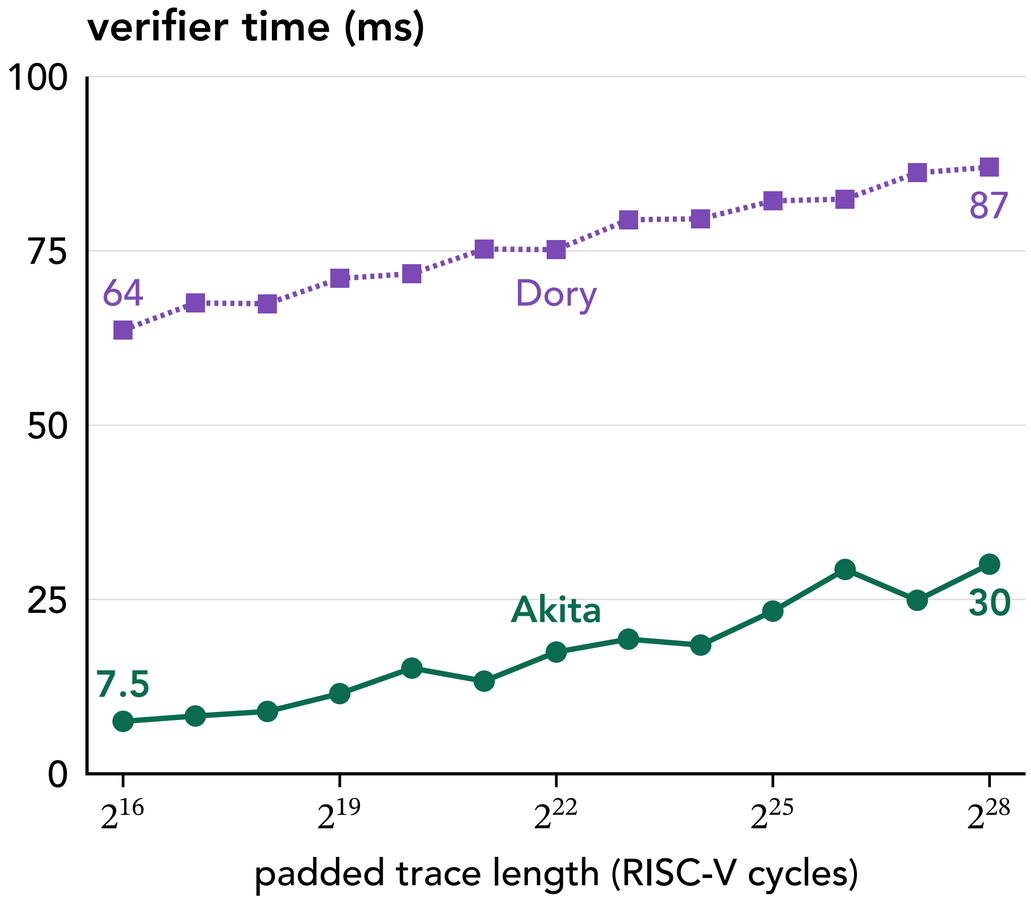
Evaluation: Jolt Verifier and Proof Size

One thread; SHA-256 chains, Apple M4 Max



Evaluation: Jolt Verifier and Proof Size

One thread; SHA-256 chains, Apple M4 Max



	verifier (ms)		proof size (KB)	
cycles	Akita	Dory	Akita	Dory
2^{16}	**7.5**	63.6	**80.5**	78.4
2^{19}	**11.5**	71.0	**84.4**	86.2
2^{22}	**17.5**	75.2	**89.5**	91.4
2^{25}	**23.3**	82.2	**91.5**	91.8
2^{28}	**30.1**	87.0	**94.1**	96.6

AI in the Making of Akita



AI in the Making of Akita



Protocol design

ideas from conversations with AI models

- embedding the evaluation claim from the field into the ring
- leaving the last block out of the final opening

AI in the Making of Akita



Protocol design

ideas from conversations with AI models

- embedding the evaluation claim from the field into the ring
- leaving the last block out of the final opening

Implementation

written, reviewed and tuned with AI

- codebase tuned for AI contributions: spec, implement, review
- autoresearch loops: commit and prove **2× faster** in two weeks

AI in the Making of Akita



Protocol design

ideas from conversations with AI models

- embedding the evaluation claim from the field into the ring
- leaving the last block out of the final opening

Implementation

written, reviewed and tuned with AI

- codebase tuned for AI contributions: spec, implement, review
- autoresearch loops: commit and prove **2× faster** in two weeks

Presentation

these slides

- each slide is a script, written by Claude (Opus 5.5)
- very fast iterations, higher-quality final results

AI in the Making of Akita



Protocol design

ideas from conversations with AI models

- embedding the evaluation claim from the field into the ring
- leaving the last block out of the final opening

Implementation

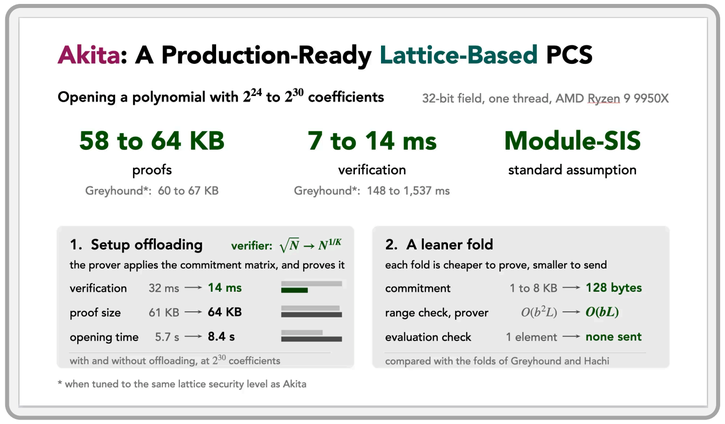
written, reviewed and tuned with AI

- codebase tuned for AI contributions: spec, implement, review
- autoresearch loops: commit and prove **2× faster** in two weeks

Presentation

these slides

- each slide is a script, written by Claude (Opus 5.5)
- very fast iterations, higher-quality final results



Summary and Future Work

Summary and Future Work

Takeaways

- setup offloading: a verifier below $\sqrt{N}$
- proofs of **58 to 64 KB**, verified in **7 to 14 ms**
- security from **Module-SIS** alone
- one-hot friendly: **2.2 to 4.2×** faster Jolt prover

Summary and Future Work

Takeaways

- setup offloading: a verifier below $\sqrt{N}$
- proofs of **58 to 64 KB**, verified in **7 to 14 ms**
- security from **Module-SIS** alone
- one-hot friendly: **2.2 to 4.2×** faster Jolt prover

Future work

- zero knowledge; binary fields [OS26]
- lattice signature aggregation [AABKT24, BBLMS26]
- verifiable OPRF: Beasley [LWZ26]
- verifiable FHE [ABPS24, WWX+25, DT26]

Summary and Future Work

Takeaways

- setup offloading: a verifier below $\sqrt{N}$
- proofs of **58 to 64 KB**, verified in **7 to 14 ms**
- security from **Module-SIS** alone
- one-hot friendly: **2.2 to 4.2×** faster Jolt prover

Future work

- zero knowledge; binary fields [OS26]
- lattice signature aggregation [AABKT24, BBLMS26]
- verifiable OPRF: Beasley [LWZ26]
- verifiable FHE [ABPS24, WWX+25, DT26]

Code
github.com/LayerZero-Labs/akita

Paper
eprint.iacr.org/2026/1983

Thank you!